

QuadSync: Quadrifocal Tensor Synchronization via Tucker Decomposition

Daniel Miao *

Gilad Lerman*

Joe Kileel[†]

Abstract

In structure from motion, quadrifocal tensors capture more information than their pairwise counterparts (essential matrices), yet they have often been thought of as impractical and only of theoretical interest. In this work, we challenge such beliefs by providing a new framework to recover n cameras from the corresponding collection of quadrifocal tensors. We form the block quadrifocal tensor and show that it admits a Tucker decomposition whose factor matrices are the stacked camera matrices, and which thus has a multilinear rank of $(4, 4, 4, 4)$ independent of n . We develop the first synchronization algorithm for quadrifocal tensors, using Tucker decomposition, alternating direction method of multipliers, and iteratively reweighted least squares. We further establish relationships between the block quadrifocal, trifocal, and bifocal tensors, and introduce an algorithm that jointly synchronizes these three entities. Numerical experiments demonstrate the effectiveness of our methods on modern datasets, indicating the potential and importance of using higher-order information in synchronization.

1. Introduction

Structure from motion (SfM) has been one of the most active research areas in 3D computer vision. SfM concerns the reconstruction of a 3D model of a scene from a set of 2D images taken from different views. Typically the pipeline of SfM includes feature detection and matching, relative pose estimation, synchronization, and reconstruction. While many works have explored synchronization based on pairwise measurements, few have investigated the possibility of incorporating higher order measurements into the SfM pipeline, even though higher order measurements may have the capability to improve reconstruction quality through stronger constraints and replicated information. One obstacle has been that higher-order measurements are less understood compared to their counterparts; there has also not been sufficient dedicated work on computing them. The authors of [1] state that, “Currently the quadrifocal and mixed trifocal tensors are useful only from a theoretical stand-point”. In this work, we lay the theoretical groundwork for synchronizing quadrifocal tensors by characterizing properties of a collection of quadrifocal tensors, while also providing two promising practical algorithms for advancing current SfM systems using quadrifocal tensors.

1.1. Relevant Previous Work

In SfM, the classical synchronization methods are incremental methods, such as Bundler [2] and COLMAP [3]. Cameras are registered sequentially while the 3D scene is reconstructed using bundle adjustment. The order in which cameras are registered may greatly impact the reconstruction quality due to error accumulation. Also, bundle adjustment becomes computationally expensive with many cameras.

Global synchronization methods have been proposed whereby cameras are processed simultaneously. Global methods can be split into different categories. Some directly process a collection of fundamental or essential matrices. This is usually achieved by enforcing algebraic constraints like low rankness [4–7] or properties of the fundamental matrix [8].

*School of Mathematics, University of Minnesota (miao0022@umn.edu, lerman@umn.edu)

[†]Department of Mathematics and Oden Institute for Computational Engineering and Sciences, University of Texas at Austin (jkileel@math.utexas.edu)

Others extract the relative rotations and relative locations from essential matrices. One can use the structure of $SO(3)$ to obtain a global estimate of the camera orientations [9–14]. Then, one can retrieve the global location configuration using the rotation estimates [15–19]. Alternatively, one may synchronize over $SE(3)$ and retrieve rotations and translations simultaneously [20–23]. Modern global-based pipelines include GLOMAP [24] and Theia [25].

Recently, there have been efforts toward investigating the use of higher-order information in synchronization, like trifocal tensors and higher-order cycles [26–28]. These works point to possible benefits of using higher-order information in SfM.

Separately, there have been attempts to utilize quadrifocal relationships in multiview geometry. [29] studies the minimal problem for estimating a special quadrifocal tensor wherein one of the views is a radial camera. [30, 31] exploit quadrifocal relationships to develop a trajectory estimation problem, yet do not directly operate on quadrifocal tensors. To the best of our knowledge, no prior works have focused on the global synchronization of a collection of quadrifocal tensors.

1.2. Our Contributions

Our contributions can be summarized as follows.

1. We develop new and strong theory: a system of algebraic constraints for a set of quadrifocal tensors. It is expressed as a low rank condition on a block tensor that we introduce.
2. We develop the first global synchronization algorithm for quadrifocal tensors, and a joint synchronization scheme that combines this with pairwise and triplewise measurements.
3. We demonstrate the promise of our algorithms through numerical experiments and show the effectiveness and practicality of using higher-order measurements in synchronization.

2. Background

Given a collection of n images $I_1, \dots, I_n$ of a 3D scene, standard pinhole cameras associated with each image are modeled as 3×4 matrices $P_i = K_i R_i [I_{3 \times 3} \mid -t_i]$, where $K_i \in \mathbb{R}^{3 \times 3}$ is the calibration matrix, $R_i \in SO(3)$ is the orientation, and $t_i \in \mathbb{R}^3$ is the location of the camera in the global coordinate system. Let $\mathbf{X} \in \mathbb{R}^4$ be a world point in homogeneous coordinates, and $\mathbf{x}_i = P_i \mathbf{X}$ its projection into the image plane of I_i in homogeneous coordinates. The fundamental matrix F_{ij} relates points $\mathbf{x}_i$ and $\mathbf{x}_j$ that correspond to the same world point in the image planes I_i and I_j respectively via

$$\mathbf{x}_i^T F_{ij} \mathbf{x}_j = 0.$$

Analogously, the trifocal tensor $T_{ijk} = \{T_1, T_2, T_3\} \in \mathbb{R}^{3 \times 3 \times 3}$ relates corresponding points across three images I_i, I_j, I_k , such that

$$[\mathbf{x}_j]_{\times} \left(\sum_{w=1}^3 (\mathbf{x}_i)_w T_w \right) [\mathbf{x}_k]_{\times} = 0_{3 \times 3},$$

where $[\cdot]_{\times}$ denotes the corresponding skew-symmetric matrix.

We also recall how global synchronization for fundamental matrices and trifocal tensors can be conducted. For synchronizing a collection of fundamental matrices, [4] introduces the *multiview matrix of fundamentals* $\mathcal{F}^n$ (or the *n-view fundamental matrix* or the *block fundamental matrix*). It is formed by concatenating the $\binom{n}{2}$ fundamental matrices associated with n cameras into a $3n \times 3n$ matrix $\mathcal{F}^n$, thought of as an $n \times n$ block matrix where the ij^{th} block is $F_{ij} \in \mathbb{R}^{3n \times 3n}$. With a suitable set of nonzero scales on F_{ij} , then $\mathcal{F}^n$ admits a factorization $\mathcal{F}^n = A + A^T$ where $\text{rank}(A) = 3$ for not all collinear cameras, so that $\text{rank}(\mathcal{F}^n) = 6$. The constraints on $\mathcal{F}^n$ were further explored in [6], [5], [7]. The full characterization of the n -view fundamental matrix from [5] is summarized by the following theorem.

Theorem 2.1 ([5]). *A given matrix $\mathcal{F}^n \in \mathbb{R}^{3n \times 3n}$ is consistent as an n -view fundamental matrix with some set of n cameras whose centers are not all collinear if and only if:*

1. $\text{rank}(\mathcal{F}^n) = 6$ and $\mathcal{F}^n$ has exactly 3 positive and 3 negative eigenvalues.
2. $\text{rank}(\mathcal{F}_{i,:}^n) = 3$ for all $i = 1, \dots, n$, where $\mathcal{F}_{i,:}^n$ denotes the i^{th} row block of $\mathcal{F}^n$.

Recently [27] explores the interesting higher-order setting with trifocal tensors. The *block trifocal tensor* $\mathcal{T}^n \in \mathbb{R}^{3n \times 3n \times 3n}$ was introduced and formed by stacking the $3 \times 3 \times 3$ trifocal tensors T_{ijk} in the ijk^{th} position in $\mathcal{T}^n$. It was shown that $\mathcal{T}^n$ admits an exact low rank Tucker decomposition when its blocks are suitably scaled. The characterization of $\mathcal{T}^n$ is as follows.

Theorem 2.2 ([27]). *The block trifocal tensor admits a Tucker factorization, $\mathcal{T}^n = \mathcal{G}_T \times_1 \mathcal{P} \times_2 \mathcal{C} \times_3 \mathcal{C}$, where $\mathcal{G}_T \in \mathbb{R}^{6 \times 4 \times 4}$, $\mathcal{P} \in \mathbb{R}^{3n \times 6}$, and $\mathcal{C} \in \mathbb{R}^{3n \times 4}$. If the n cameras that produce $\mathcal{T}^n$ are not all collinear, then $\text{mlrank}(\mathcal{T}^n) = (6, 4, 4)$. Otherwise, $\text{mlrank}(\mathcal{T}^n) \preceq (6, 4, 4)$.*

2.1. Quadrifocal Tensors

Analogous to fundamental matrices and trifocal tensors, quadrifocal tensors encode information across four views. In tensor notation, for views I_i, I_j, I_k, I_l , corresponding image points are related through the quadrifocal tensor $Q_{ijkl} \in \mathbb{R}^{3 \times 3 \times 3 \times 3}$ via

$$\mathbf{x}_i^a \mathbf{x}_j^b \mathbf{x}_k^c \mathbf{x}_l^d \epsilon_{apw} \epsilon_{bqx} \epsilon_{cry} \epsilon_{dsz} (Q_{ijkl})^{pqrs} = 0_{wxyz},$$

where ϵ is the Levi-Civita tensor and there are implicit summations over repeated indices. Quadrifocal tensors also relate corresponding image lines or mixtures of lines and points on lines. From the camera matrices P_i, P_j, P_k, P_l , the corresponding quadrifocal tensor Q_{ijkl} can be calculated directly through

$$(Q_{ijkl})^{pqrs} = \det \begin{bmatrix} P_i^p \\ P_j^q \\ P_k^r \\ P_l^s \end{bmatrix}. \quad (1)$$

Like trifocal tensors and fundamental matrices, quadrifocal tensors are only well-defined up to nonzero scale since the camera matrices P_i are themselves only well-defined up to nonzero scales. We will need to find appropriate scales below.

There are certain interesting characteristics of quadrifocal tensors that motivate our choice to use them for synchronization. Firstly, quadrifocal tensors capture the interactions of four views, and thus encode complex geometric information. The quadrifocal tensor includes triple and pairwise information too. [27] showed the effectiveness of using redundant information in trifocal tensors for better averaging of translations, and quadrifocal tensors stand to further improve results. Secondly, there may be more flexibility for estimating quadrifocal tensors, especially in homogeneous scenes. In line correspondence relationships of quadrifocal tensors, the lines in the image planes do not necessarily have to correspond to the same world line. The quadrifocal tensor can also be estimated from 6 point correspondences. We refer to the following for detecting and matching lines across views [32, 33]. Thirdly, in the case of quadrifocal tensors all views are treated equally, unlike in the trifocal tensor case where there is a view treated specially. This might lead to a more stable algorithm in synchronization. Lastly, but not least, estimating a quadrifocal tensor requires sufficiently many inlier features across four views. This implicitly enforces consistency of the viewing graph through matched points and/or lines. Intentionally using quadrifocal tensors for synchronization may therefore help us to start synchronization from a cleaner graph, especially in the dense viewing graph case. We refer to [34–37] for more on the properties and estimation of quadrifocal tensors. We also refer to [38–42] for more on solving minimal problems related to computer vision. We refer to [36, 42–45] for results on minimal problems and the characterization of higher order tensors from an algebraic geometry perspective.

2.2. Tucker Decomposition

In this section, we review the Tucker decomposition and the multilinear rank of a tensor. Let $\mathcal{X} \in \mathbb{R}^{M_1 \times M_2 \times \dots \times M_N}$ be an order N tensor, i.e., an N dimensional array. The mode- i flattening (or matricization) $\mathcal{X}_{(i)} \in \mathbb{R}^{M_i \times (\prod_{j \neq i} M_j)}$ is the rearrangement of $\mathcal{X}$ into a matrix by taking the slices along the i -th mode of the tensor to form rows of a matrix in lexicographic order. The Frobenius norm of a tensor is defined as $\|\mathcal{X}\| = (\sum_{i_1, \dots, i_N} \mathcal{X}_{i_1 i_2 \dots i_N}^2)^{1/2} = \|\mathcal{X}_{(i)}\|_F$. Let R_i denote

the matrix rank of $\mathcal{X}_{(i)}$. Then the multilinear rank of $\mathcal{X}$ is $\text{mlrank}(\mathcal{X}) = (R_1, R_2, \dots, R_N)$. The i -th mode tensor-matrix product $\times_i$ is multiplication between tensor $\mathcal{X}$ and $U \in \mathbb{R}^{m \times M_i}$ is $\mathcal{X} \times_i U \in \mathbb{R}^{M_1 \times \dots \times M_{i-1} \times m \times M_{i+1} \times \dots \times M_N}$ given by

$$(\mathcal{X} \times_i U)_{j_1 \dots j_{i-1} k j_{i+1} \dots j_N} = \sum_{j_i=1}^{M_i} \mathcal{X}_{j_1 j_2 \dots j_N} U_{k j_i}.$$

A Tucker decomposition $\mathcal{X}$ is any tensor factorization of the following form

$$\mathcal{X} = \mathcal{G} \times_1 U_1 \times_2 U_2 \times_3 \dots \times_N U_N =: \llbracket \mathcal{G}; U_1, U_2, \dots, U_N \rrbracket,$$

where $\mathcal{G} \in \mathbb{R}^{R_1 \times R_2 \times \dots \times R_N}$ is a core tensor, and $U_i \in \mathbb{R}^{I_i \times R_i}$ are called factor matrices. One standard way to obtain a Tucker decomposition for $\mathcal{X}$ is the higher-order singular value decomposition (HOSVD). In HOSVD, the factor matrices U_i are the R_i leading left singular vectors of $\mathcal{X}_{(i)}$. The core tensor can then be obtained as $\mathcal{G} = \mathcal{X} \times_1 U_1^T \times_2 \dots \times_N U_N^T$. Similar to the matrix case, HOSVD can be used to project onto the set of lower multilinear rank tensors $A = \{\mathcal{X} \in \mathbb{R}^{M_1 \times M_2 \times \dots \times M_N} \mid \text{mlrank}(\mathcal{X}) = (r_1, r_2, \dots, r_N)\}$ for $r_i \leq R_i$. Then, HOSVD will satisfy a quasi-optimal property, where if $X^* = \arg\min_{X \in A} \|X - \mathcal{X}\|_F$ and X' is the tensor obtained from truncating HOSVD to the ranks $r_1, r_2, \dots, r_n$, then

$$\|\mathcal{X} - X'\|_F \leq \sqrt{N} \|\mathcal{X} - X^*\|_F.$$

We refer to [46] for more information on tensor decomposition.

3. The Block Quadrifocal Tensor

We first introduce the block quadrifocal tensor, which is a novel construction. Given n cameras $\{P_i\}_{i=1}^n$, we form the *block quadrifocal tensor* $\mathcal{Q}^n \in \mathbb{R}^{3n \times 3n \times 3n \times 3n}$ by stacking the quadrifocal tensors along the four modes. Thus, the $ijkl^{\text{th}}$ block of $\mathcal{Q}^n$ is the quadrifocal tensor $Q_{ijkl} \in \mathbb{R}^{3 \times 3 \times 3 \times 3}$ corresponding to the cameras P_i, P_j, P_k, P_l calculated using equation (1).

3.1. Properties of the Block Quadrifocal Tensor

Next, we develop theory for the block quadrifocal tensor $\mathcal{Q}^n$. We establish interesting properties of it, including low multilinear rank, low projection rank, a simple relationship to camera poses, and the sufficiency of the low multilinear rank to determine the scales of each block in $\mathcal{Q}^n$. All proofs are supplied in the supplementary materials. We start with the following important characterization of the block quadrifocal tensor, in terms of Tucker decomposition and multilinear rank.

Theorem 3.1. *Let $\mathcal{Q}^n$ be a block quadrifocal tensor. Then there a choice of blockwise nonzero scales such that $\mathcal{Q}^n = \mathcal{G}_Q \times_1 C \times_2 C \times_3 C \times_4 C$, where $C \in \mathbb{R}^{3n \times 4}$ is the stacked camera matrix and $\mathcal{G}_Q \in \mathbb{R}^{4 \times 4 \times 4 \times 4}$ is a constant sparse tensor with all entries in $\{-1, 0, 1\}$. When the cameras that produce $\mathcal{Q}^n$ do not all share the same camera center, then $\text{mlrank}(\mathcal{Q}^n) = (4, 4, 4, 4)$.*

Remark 1. *The last sentence of the theorem shows an advantage of $\mathcal{Q}^n$ over the n -view fundamental matrix $\mathcal{F}^n$ and the block trifocal tensor $\mathcal{T}^n$. Both $\mathcal{F}^n$ and $\mathcal{T}^n$ experience a rank drop as soon as the corresponding cameras are collinear. In that case, $\text{rank}(\mathcal{F}^n) = 4$ (see [7]) and $\text{mlrank}(\mathcal{T}^n) = (5, 4, 4)$ (shown in the supplementary materials). Synchronizing based on fundamental matrices and trifocal tensors then requires additional procedures like constructing virtual cameras.*

Given the explicit Tucker factorization of the block quadrifocal tensor $\mathcal{Q}^n$, there is a simple procedure to retrieve cameras from $\mathcal{Q}^n$ up to an invertible 4×4 linear transformation, or a projective transformation of world coordinates.

Corollary 3.1.1 (Projective Reconstruction). *Given $\mathcal{Q}^n$ with appropriately chosen blockwise scales, one can retrieve the n camera matrices up to a global projective frame ambiguity, through the higher-order singular value decomposition. One simply takes the 4 singular vectors corresponding to the largest singular values in any flattening.*

Remark 2. Intuitively, Theorem 3.1 gives a stronger constraint compared to the two view and three view case, and thereby shows an advantage of using higher-order measurements for synchronization. Roughly speaking, valid block quadrifocal tensors are restricted to a relatively smaller subset inside the full tensor space, as compared to their counterparts. This can be seen by counting the codimensions of the sets of low rank tensors, where Q, T, E here will represent the set of low Tucker rank tensors/matrices for quadrifocal, trifocal, and essential matrices, respectively. Here, $\text{codim}_{(\mathbb{R}^{3n})^{\otimes 4}}(Q) > \text{codim}_{(\mathbb{R}^{3n})^{\otimes 3}}(T) > \text{codim}_{\mathbb{R}^{3n \times 3n}}(E)$. A degrees of freedom count shows $\text{codim}_{(\mathbb{R}^{3n})^{\otimes 4}}(Q) = \Omega(n^4)$, $\text{codim}_{(\mathbb{R}^{3n})^{\otimes 3}}(T) = \Omega(n^3)$, and $\text{codim}_{\mathbb{R}^{3n \times 3n}}(E) = \Omega(n^2)$.

The paper [44] uses the projection rank (P-Rank) of a single trifocal tensor. We recall the definition of the projection rank of a tensor and show that the block quadrifocal tensor has a low projection rank independent of n . We also establish the projection rank for the block trifocal tensor later in the paper.

Definition 1. The projection rank is defined as the tuple of ranks of the various projections from an order n tensor to matrices by contraction with any generic tuple of $n-2$ vectors, where $n \geq 3$.

For example, given $Q = \mathcal{Q}^n$, its projection rank is a tuple of 6 numbers, corresponding to the ranks of the matrices $\sum_{i,j} x_i y_j Q_{ij::}$, $\sum_{i,j} x_i y_j Q_{i:j}$, $\sum_{i,j} x_i y_j Q_{i::j}$, $\sum_{i,j} x_i y_j Q_{:ij}$, $\sum_{i,j} x_i y_j Q_{:i:j}$, $\sum_{i,j} x_i y_j Q_{::ij}$ for generic vectors $x_i, y_j \in \mathbb{R}^{3n}$.

Theorem 3.2. Let $\mathcal{Q}^n$ be a block quadrifocal tensor with appropriately chosen blockwise scales. Then $\text{P-Rank}(\mathcal{Q}^n) = (2, 2, 2, 2, 2, 2)$.

Note that this is not implied by the low multilinear rank; it depends on the structure of the core tensor in Theorem 3.1. By optimizing with the fixed core, and preserving the symmetries in the factor matrices, we will enforce the P-Rank constraint.

The block quadrifocal tensor contains explicit information of the two view and three view geometry through the next result.

Proposition 3.3. The block quadrifocal tensor $Q = \mathcal{Q}^n$ admits the following properties.

1. The super-diagonal blocks vanish, $Q_{iiii} = 0$ for all $i = 1, \dots, n$.
2. The blocks where three of the indices are the same correspond to the camera center of view i in view j .
3. The blocks where two of the indices are the same correspond to elements in the trifocal tensor T_{ijk} .
4. The blocks where two of the indices are the same and the other two are also the same correspond to elements in the fundamental matrix F_{ij} .

Lastly, we stress that quadrifocal tensors can only be estimated up to an unknown nonzero scale. The nice theoretical properties rely on choosing a suitable set of scales. Thus it is important to understand how our constraints determine the unknown scales. In the following theorem, the sufficiency of the low multilinear rank constraint to determine the scales on the block quadrifocal tensor is established, so that camera poses can be uniquely determined. This theorem supports our algorithm development in the next section. The notation $\odot_b$ denotes blockwise multiplication.

Theorem 3.4. Let $\mathcal{Q}^n \in \mathbb{R}^{3n \times 3n \times 3n \times 3n}$ be a block quadrifocal tensor corresponding to $n \geq 5$ generic cameras. Let $\Lambda \in \mathbb{R}^{n \times n \times n \times n}$ be a block scaling with $\lambda_{ijkl} \neq 0$ if and only if i, j, k, l are not all equal. If $\Lambda \odot_b \mathcal{Q}^n \in \mathbb{R}^{3n \times 3n \times 3n \times 3n}$ has multilinear rank $(4, 4, 4, 4)$, then there exists $\alpha, \beta, \gamma, \delta \in \mathbb{R}^n$ such that $\lambda_{ijkl} = \alpha_i \beta_j \gamma_k \delta_l$ whenever i, j, k, l are not all the same. That is, Λ and $\alpha \otimes \beta \otimes \gamma \otimes \delta$ are equal away from the super diagonal.

4. Method

The input to our synchronization method will be a collection of estimated quadrifocal, trifocal tensors, and fundamental matrices. There may be many missing measurements, and we assume all missing blocks are filled in with zeros. Let $\Omega = \{(i, j, k, l) \mid Q_{ijkl} \neq 0_{3 \times 3 \times 3 \times 3} \text{ or } i = j = k = l\}$ denote the set of indices that are observed. The estimated block quadrifocal tensor $\mathcal{Q}^n$ is formed by stacking the blocks. Recall that quadrifocal tensors are defined up to scale and each

estimated quadrifocal tensors will be associated with an unknown scale. Assuming that the true underlying block quadrifocal tensor is $\mathcal{Q}^n$, the estimated block quadrifocal tensor is $\tilde{\mathcal{Q}}^n = P_\Omega(\Lambda \odot_b \mathcal{Q}^n)$, where $\odot_b$ denotes block multiplication and P_Ω is the projection onto observed blocks. Let $S^4(\mathbb{R}^n)$ denote the set of symmetric tensors of size n^4 and let Λ denote the variable of unknown scales. Note that by 1, we have the fact that

$$\mathcal{Q}_{\pi(ijkl)}^n = \text{sgn}(\pi) \mathcal{Q}_{ijkl}^n, \quad (2)$$

where π is a permutation of the indices. Thus, we can restrict Λ to be in the set of symmetric tensors $S^4(\mathbb{R}^n)$, since the elements in $\mathcal{Q}_{\pi(ijkl)}$ will have the same value up to a sign in $\mathcal{Q}_{ijkl}$.

4.1. QuadSync: An ADMM-IRLS Scheme

We wish to solve for the scales and the camera matrices given the estimated block quadrifocal tensor $\tilde{\mathcal{Q}}^n$. We form the following optimization problem

$$\begin{aligned} \min_{\Lambda, C} \quad & \sum_{(i,j,k,l) \in \Omega} \|\Lambda_{ijkl}(\tilde{\mathcal{Q}}^n)_{ijkl} - [\mathcal{G}_Q; C, C, C, C]_{ijkl}\|_F \\ \text{s.t.} \quad & \Lambda \in S^4(\mathbb{R}^n) \text{ and } \|\Lambda\|_F^2 = 1, \end{aligned}$$

The condition of $\|\Lambda\|_F^2 = 1$ is enforced to avoid a trivial solution. The norms are added without squaring them to reduce sensitivity to outliers. The problem is difficult to solve due to the quartic degree with respect to C and the nonconvexity from the scales. By introducing an auxiliary variable $B \in \mathbb{R}^{3n \times 4}$, the problem is equivalent to the following constrained optimization problem where the factors are separated,

$$\begin{aligned} \min_{\Lambda, C_i, B} \quad & \sum_{(i,j,k,l) \in \Omega} \|\Lambda_{ijkl}(\tilde{\mathcal{Q}}^n)_{ijkl} - [\mathcal{G}_Q; C_1, C_2, C_3, C_4]_{ijkl}\|_F \\ \text{s.t.} \quad & C_1 = C_2 = C_3 = C_4 = B \\ & \Lambda \in S^4(\mathbb{R}^n) \text{ and } \|\Lambda\|_F^2 = 1. \end{aligned} \quad (3)$$

We then form the augmented Lagrangian and solve the problem through ADMM with scaled dual variables:

$$\begin{aligned} \max_{\Gamma_i} \min_{\Lambda, C_i, B} \quad & \sum_{(i,j,k,l) \in \Omega} \|\Lambda_{ijkl}(\tilde{\mathcal{Q}}^n)_{ijkl} - [\mathcal{G}_Q; C_1, C_2, C_3, C_4]_{ijkl}\|_F \\ & + \frac{\rho}{2} \sum_{i=1}^4 \|C_i - B + \Gamma_i\|_F^2 \quad \text{s.t. } \Lambda \in S^4(\mathbb{R}^n) \text{ and } \|\Lambda\|_F^2 = 1. \end{aligned}$$

4.1.1. IRLS Outer Loop

To solve the L_1 norm optimization in the main loss function, we use Iterative Reweighted Least Squares (IRLS).

In each IRLS iteration, we calculate the current set of weights, and then solve for all the other variables. Solving for the other variables is a least squares problem. Specifically, the problem becomes

$$\begin{aligned} \max_{\Gamma_i} \min_{\Lambda, C_i, B} \quad & \|W \odot_b ((\Lambda \odot_b \tilde{\mathcal{Q}}^n) - [\mathcal{G}_Q; C_1, C_2, C_3, C_4])\|_F^2 + \\ & \frac{\rho}{2} \sum_{i=1}^4 \|C_i - B + \Gamma_i\|_F^2 \quad \text{s.t. } \Lambda \in S^4(\mathbb{R}^n) \text{ and } \|\Lambda\|_F^2 = 1. \end{aligned} \quad (4)$$

Here W is the set of IRLS weights included in the norm, so

$$w_{ijkl} = \begin{cases} 1/m_{ijkl}^t & \text{if } (i,j,k,l) \in \Omega \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

and

$$m_{ijkl}^t = \max(\delta, \sqrt{\|\Lambda_{ijkl}^{(t-1)} \tilde{\mathcal{Q}}_{ijkl}^n - \mathcal{G}_Q \times_1 (C_1)^{(t-1)} \times_2 (C_2)^{(t-1)} \times_3 (C_3)^{(t-1)} \times_4 (C_4)^{(t-1)}\|_F}).$$

using the variables from the previous IRLS iteration. δ is a regularization parameter that upper-bounds the weights.

4.1.2. ADMM Inner Loop

After fixing the weights for the outer IRLS loop, we solve the weighted least squares in (4) for $\Gamma_i, \Lambda, C_i, B$ with Alternating Direction Method of Multipliers (ADMM).

- (1) $C_1, C_2, C_3, C_4, \Lambda$: We first solve for Λ and the factors C_i through alternating minimization. To solve for C_i 's, given the convexity of the problem when fixing other variables, we can explicitly calculate the derivative and solve for C_i by setting it to zero. Note that the flattening of the tensor can be written in terms of the factor matrices and the core from the Tucker factorization. For example, $\mathcal{Q}_{(1)}^n = C_1(\mathcal{G}_Q)_{(1)}(C_4 \otimes C_3 \otimes C_2)^T$. We use $C_{\sim i}$ to denote the matrix from Kronecker products, so that for the i th mode, $C_{\sim i} = C_4 \otimes \dots \otimes C_{i+1} \otimes C_{i-1} \otimes \dots \otimes C_1$. Let $K = (\mathcal{G}_Q)_{(i)} C_{\sim i}^T$. Then,

$$\frac{\rho}{2} C_i + (W_{(i)}^2 \odot_b [C_i K]) K^T = \frac{\rho}{2} (B - \Gamma_i) + (W_{(i)}^2 \odot_b (\Lambda \odot_b \tilde{\mathcal{Q}}^n)_{(i)}) K^T.$$

We can solve for each row of C_i separately. Letting x_j denote the j th row of C_i , we have

$$x_j (\frac{\rho}{2} I_{4 \times 4} + K \text{diag}((W_{(i)}^2)_j) K^T) = \frac{\rho}{2} (B - \Gamma_i)_j + ((W_{(i)}^2)_j \odot_b [(\Lambda \odot_b \tilde{\mathcal{Q}}^n)_{(i)}]_j) K^T$$

and

$$x_j = (\frac{\rho}{2} (B - \Gamma_i)_j + ((W_{(i)}^2)_j \odot_b [(\Lambda \odot_b \tilde{\mathcal{Q}}^n)_{(i)}]_j) K^T) (\frac{\rho}{2} I_{4 \times 4} + K \text{diag}((W_{(i)}^2)_j) K^T)^{-1} \quad (6)$$

where $\text{diag}((W_{(i)}^2)_j)$ is the diagonal matrix corresponding to $((W \otimes 1_{3 \times 3 \times 3 \times 3})_{(i)}^2)_j$.

Remark 3. Solving for C_i via (6) can be slow, as the dimensions for K are $4 \times 27n^3$, and $\text{diag}(W_j^2)$ is $27n^3 \times 27n^3$. Though it is sparse, it can become computationally expensive. Each row can be solved in parallel to speed up computations. In addition, one can easily apply randomized updates to speed up the algorithm. One can randomly sample m columns, so that the multiplication becomes of size $4 \times m$ and $m \times m$ and $m \times 4$. Since the low-rankness is independent of the number of cameras, we can take $m = O(1)$. We include a small experiment in the appendices, showing that speed-ups can be achieved without sacrificing accuracy this way.

For Λ , we can solve by directly solving the convex optimization problem:

$$\begin{aligned} \min_{\Lambda} \quad & \|W \odot_b [(\Lambda \odot_b \tilde{\mathcal{Q}}^n) - \llbracket \mathcal{G}_Q; C_1, C_2, C_3, C_4 \rrbracket]\|_F^2 \\ \text{s.t.} \quad & \Lambda \in S^4(\mathbb{R}^n) \text{ and } \|\Lambda\|_F^2 = 1. \end{aligned} \quad (7)$$

This is done by solving for each block separately, where

$$\Lambda_{ijkl} = \frac{\text{trace}((\llbracket \mathcal{G}_Q; C_1, C_2, C_3, C_4 \rrbracket_{ijkl})_{(1)}^T ((\tilde{\mathcal{Q}}^n)_{ijkl})_{(1)})}{\|((\tilde{\mathcal{Q}}^n)_{ijkl})_{(1)}\|_F^2}.$$

After calculating Λ , we first symmetrize, then normalize Λ so that $\|\Lambda\|_F^2 = 1$ and $\Lambda \in S^4(\mathbb{R}^n)$. We iterate this process for solving for C_i, Λ iteratively for 10 times.

(2) $\mathbf{B}$: For B , we can solve directly, via

$$\frac{1}{4} \left(\sum_{i=1}^4 C_i + \Gamma_i \right) = B. \quad (8)$$

(3) Γ_i : For the final ascent step, we set

$$\Gamma_i^{(k+1)} = \Gamma_i^{(k)} + (C_i - B). \quad (9)$$

4.2. A Joint Optimization Framework

We also develop a joint optimization framework that simultaneously synchronizes the block quadrifocal tensor, the block trifocal tensor, and the block essential matrix. We develop it only for the calibrated case. We observe that the block quadrifocal tensor and the block trifocal tensor share same factor matrices, i.e., the $3n \times 4$ stacked camera matrices. Let $\mathcal{P}$ denote the $3n \times 6$ matrix of the line projection matrices associated with each camera matrix. See the appendices for more details on the line projection matrices. We establish a new factorization of the block essential matrix, and show that it shares factor matrices with the block trifocal tensor in the first mode in the following result.

Theorem 4.1. *Let $\mathcal{E}^n \in \mathcal{S}^2(\mathbb{R}^{3n})$ be a block essential matrix corresponding to the calibrated cameras $C \in \mathbb{R}^{3n \times 4}$. Then $\mathcal{E}^n$ admits the factorization:*

$$\mathcal{E}^n = \mathcal{P} \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \mathcal{P}^T = \mathcal{P} \mathcal{G}_E \mathcal{P}^T. \quad (10)$$

We next establish the projection rank for the block trifocal tensor, which will also be implicitly enforced.

Theorem 4.2. *Let $\mathcal{T}^n \in \mathbb{R}^{3n \times 3n \times 3n}$ be a block trifocal tensor. Then $P\text{-Rank}(\mathcal{T}^n) = (4, 3, 3)$.*

We denote the core matrix here as $\mathcal{G}_E$. Since the block trifocal tensor $\mathcal{T}^n$ and block quadrifocal tensor $\mathcal{Q}^n$ can be factorized as $\mathcal{T}^n = \llbracket \mathcal{G}_T; \mathcal{P}, C, C \rrbracket$ and $\mathcal{Q}^n = \llbracket \mathcal{G}_Q; C, C, C, C \rrbracket$, we can formulate the following optimization problem to synchronize all three entities simultaneously:

$$\begin{aligned} \min_{\Lambda_E, \Lambda_T, \Lambda_Q, C} \quad & \frac{1}{n_Q} \|W_Q \odot_b (\Lambda_Q \odot_b \tilde{\mathcal{Q}}^n - \llbracket \mathcal{G}_Q; C, C, C, C \rrbracket)\|_F^2 + \\ & \frac{1}{n_T} \|W_T \odot_b (\Lambda_T \odot_b \tilde{\mathcal{T}}^n - \llbracket \mathcal{G}_T; \mathcal{P}, C, C \rrbracket)\|_F^2 + \\ & \frac{1}{n_E} \|W_E \odot_b (\Lambda_E \odot_b \tilde{\mathcal{E}}^n - \llbracket \mathcal{G}_E; \mathcal{P}, \mathcal{P} \rrbracket)\|_F^2 \\ \text{s.t.} \quad & \|\Lambda_Q\|_F^2 = \|\Lambda_T\|_F^2 = \|\Lambda_E\|_F^2 = 1, \\ & \Lambda_Q \in S^4(\mathbb{R}^n), \Lambda_T(i, :, :), \Lambda_E \in S^2(\mathbb{R}^n), \end{aligned}$$

where n_Q, n_T, n_E are the number of estimated blocks in the estimated block quadrifocal tensor $\tilde{\mathcal{Q}}^n$, the estimated block trifocal tensor $\tilde{\mathcal{T}}^n$, and the estimated block essential matrix $\tilde{\mathcal{E}}^n$, respectively. After separating factors, let us denote the three cost functions as $f_Q(W_Q, \Lambda_Q, C_1, C_2, C_3, C_4)$, $g_T(W_T, \Lambda_T, P_1, C_5, C_6)$, $h_E(W_E, \Lambda_E, P_2, P_3)$, respectively. We also have IRLS weights set for W_q, W_t, W_e defined analogously to (5). See the appendices for more.

4.2.1. Joint Opt. with ADMM-IRLS

Similarly, we solve the joint optimization problem with an ADMM formulation:

$$\min_{\Lambda_E, \Lambda_T, \Lambda_Q, C} f_Q(W_Q, \Lambda_Q, C_1, C_2, C_3, C_4) + g_T(W_T, \Lambda_T, P_1, C_5, C_6) + h_E(W_E, \Lambda_E, P_2, P_3) + \frac{\rho}{2} \sum_{i=1}^6 \|C_i - B + \Gamma_i\|_F^2 + \frac{\rho}{2} \sum_{i=1}^3 \|P_i - D + \tau_i\|_F^2$$

such that $\|\Lambda_Q\|_F^2 = \|\Lambda_T\|_F^2 = \|\Lambda_E\|_F^2 = 1, \Lambda_Q \in S^4(\mathbb{R}^n)$, $\Lambda_T(i, :, :)$ and $\Lambda_E \in S^2(\mathbb{R}^n)$.

- (1) **C_i, P_i:** We first aim to solve for the C_i 's and P_i 's. The updates for C_1, C_2, C_3, C_4 are the same as before in the quadrifocal tensor case but with additional weights n_Q . For C_5, C_6 , the derivation is identical. To solve for C_i , let $C_{\sim i} = P \otimes C_j$, and let $K = (\mathcal{G}_T)_{(i)} C_{\sim i}^T$. Then, we have the update rule for C_i as

$$x_j = \left(\frac{\rho}{2} (B - \Gamma_i)_j + \frac{1}{n_T} ((W_T^2 \odot L_T) \odot_b T)_j K^T \right) \left(\frac{\rho}{2} I_{4 \times 4} + \frac{1}{n_T} K \text{diag}((W_T)_j^2) K^T \right)^{-1}.$$

For P_1 , let $K = (\mathcal{G}_T)_{(3)} (C_2 \otimes C_1)^T$. Then

$$x_j = \left(\frac{\rho}{2} (D - \tau)_j + \frac{1}{n_T} ((W_T^2 \odot L_T) \odot_b T)_j K^T \right) \left(\frac{\rho}{2} I_{4 \times 4} + \frac{1}{n_T} K \text{diag}(W_T^2) K^T \right)^{-1}.$$

Similarly, we also solve for each row separately for P_2, P_3 with $\Lambda_E, W_E, \mathcal{G}_E$ and n_E .

- (2) **$\Lambda_Q, \Lambda_T, \Lambda_E, B, D, \Gamma_i, \tau_i$:** They are all the same or analogous as in (7), (8), (9). The order of the optimization is $C_1, C_2, C_3, C_4, C_5, C_6, P_1, P_2, P_3, \Lambda_Q, \Lambda_T, \Lambda_E$ alternatively for a couple iterations. Then B, D , then Γ_i, τ_i . After the solving the ADMM step, we recalculate the IRLS weights and repeat until convergence. More details of the algorithm can be found in the appendices.

5. Experiments

5.1. A Higher-Order Cycle Based Heuristic

We develop a heuristic to estimate the corruption level of a quadruplet of cameras using trifocal tensors. Similar consistency measurements for group synchronization have been proposed in [28], but here our objects are camera matrices and do not admit a group structure since the matrices are non-square.

Given a quadruplet of images I_i, I_j, I_k, I_l , suppose we have four trifocal tensor estimates $T_{ijk}, T_{jkl}, T_{kli}, T_{lij}$. Assuming each trifocal tensor is consistent, we have camera matrices corresponding to each trifocal tensor. Specifically, suppose T_{ijk} gives $\{P_i^1, P_j^1, P_k^1\}$, T_{jkl} gives $\{P_j^2, P_k^2, P_l^2\}$, T_{kli} gives $\{P_k^3, P_l^3, P_i^3\}$, and T_{lij} gives $\{P_l^4, P_i^4, P_j^4\}$, where each set of cameras is in a different projective frame. Given two sets of the same cameras P_j^1, P_k^1 and P_j^2, P_k^2 in different projective frames, they can be synchronized by solving for a projective transformation $H_{21} \in \mathbb{R}^{4 \times 4}$ such that $P_j^2 H_{21} = a_j P_j^1$ and $P_k^2 H_{21} = a_k P_k^1$. Then, $\{P_j^2 H_{21}, P_k^2 H_{21}, P_l^2 H_{21}\}$ is in the same frame as $\{P_i^1, P_j^1, P_k^1\}$. This can be solved for via linear least squares in the variables H_{21}, a_j, a_k . Denote this operation by $\{P_i^1, P_j^1, P_k^1\} \xleftarrow{H_{21}} \{P_j^2, P_k^2, P_l^2\}$. The following operations can then be performed:

$$\begin{aligned} & \{P_i^1, P_j^1, P_k^1\} \xleftarrow{H_{21}} \{P_j^2, P_k^2, P_l^2\} \\ & \{P_j^2 H_{21}, P_k^2 H_{21}, P_l^2 H_{21}\} \xleftarrow{H_{32}} \{P_k^3, P_l^3, P_i^3\} \\ & \{P_k^3 H_{32}, P_l^3 H_{32}, P_i^3 H_{32}\} \xleftarrow{H_{43}} \{P_l^4, P_i^4, P_j^4\}. \end{aligned}$$

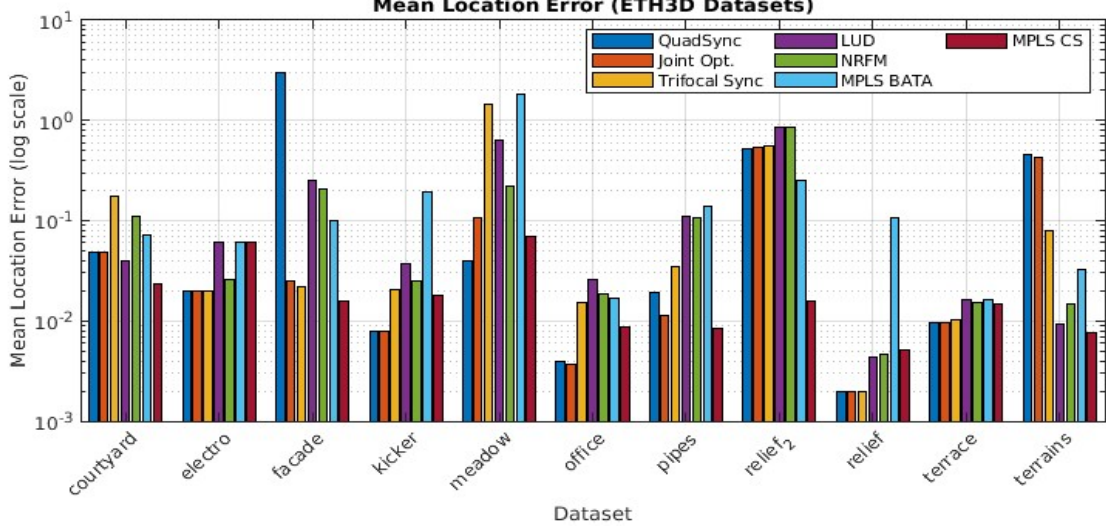


Figure 1. Mean location error for ETH3D datasets

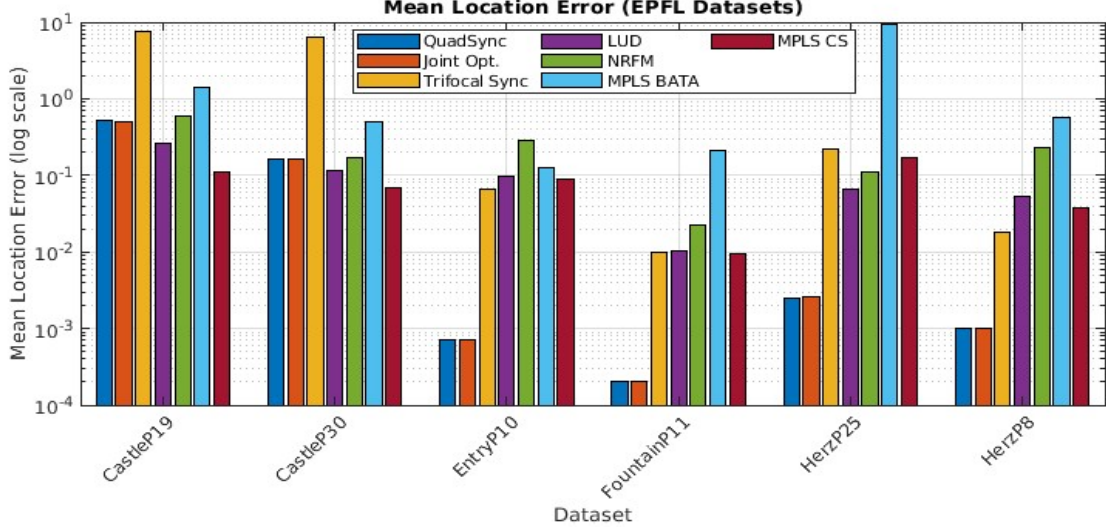


Figure 2. Mean location error for EPFL datasets

When there is no noise in any of the initial trifocal tensors, $aP_i^1 = P_i^4 H_{43}$ and $bP_j^1 = P_j^4 H_{43}$ for some nonzero scales $a, b \in \mathbb{R}$. However, when there is noise, this will not hold. Our heuristic $d(i, j, k, l)$ can be defined as a tuple measuring the distance between the first two and last two cameras:

$$d(i, j, k, l) := g(d(P_i^1, P_i^4 H_{43}), d(P_j^1, P_j^4 H_{43})). \quad (11)$$

In our experiments, we choose $d(P_m, P_n)$ as either the angle between the orientation of calibrated cameras P_m, P_n or the relative difference of the locations. Further, we choose $g(\cdot, \cdot)$ to be the average.

5.2. Estimation of Quadrifocal Tensors

Given images I_i with $i = 1, \dots, N$, the procedure of estimating the quadrifocal tensors is described below.

1. First, estimate trifocal tensors from point correspondences. We follow the procedure in [27], where the Subspace Con-

strained Tyler’s M-estimator (STE) from [47] is used to reject outliers, assuming at most 40% of point correspondences are inliers. Initial trifocal tensors are estimated linearly with at most 30 inlier points and refined with bundle adjustment. Note that the corresponding trifocal tensor T_{ijk} can also be estimated using the essential matrices E_{ji} , E_{ki} , E_{kj} . A detailed description of the latter method can be found in [34].

2. Then, enumerate all four cycles in this viewing graph and calculate the inconsistency heuristic (11). Cycles whose heuristic in terms of rotation is larger than 3 degrees or is larger than 0.2 in terms of translation are neglected. For good cycles, we simply take an average of all the camera pose estimates obtained by calculating the heuristic. The quadrifocal tensor is then estimated via (1).

After estimation of the quadrifocal tensors by the above procedure, we form the estimated block quadrifocal tensor $\tilde{Q}^n$. We also fill in the blocks with two overlapping indices, which correspond to the trifocal tensors. We normalize each block in $\tilde{Q}^n$ to have Frobenius norm 1.

Remark 4. *There is a lack of practical work for estimating quadrifocal tensors. The current procedure for estimating quadrifocal tensors introduces extra sources of error. Yet, numerical results show resilience and robustness against these extra sources of noise. Our work also motivates future research on the estimation of quadrifocal tensors.*

5.3. Initialization of Algorithm

For QuadSync, we initialize the following variables W_Q , Λ_Q , C_i , B , Γ_i . We set $\rho=0.01$, run IRLS for 4 times, and use just 1 ADMM loop inside each IRLS.

- C_i , B : For the initialization of the camera matrices, we first retrieve the camera matrices from $\tilde{Q}^n$. That is, we apply the higher order singular value decomposition and simply take the first four singular vectors from any mode as the initial estimate for the cameras $\tilde{C}$. We set $C_1=C_2=C_3=C_4=B=\tilde{C}$.
- Λ_Q , W_Q : We solve (7) for the optimal set of scales. We then initialize W_Q with (5).
- Γ_i : We set $\Gamma_i^0=0_{3n \times 4}$ for all $i \in \{1,2,3,4\}$.

For the joint optimization, the variables are all initialized similarly, but ρ is set to 0.00001, and we only run 2 IRLS loops.

Remark 5. *Though our optimization problems are nonconvex, an initialization with HOSVD appears to be sufficient based on empirical evidence. This is an advantage, as two view methods such as [4], which also solve complicated nonconvex problems, usually rely on initializing with specialized and more complex methods.*

5.4. Numerical Experiments

Our method operates on datasets that are dense, such that forming the higher order 4-uniform viewing hypergraphs does not disconnect the viewing graph or cause the viewing graph to become too sparse. We truncate all datasets so that they satisfy a density requirement, as follows. We first obtain all the indices $(i,j,k,l) \in \Omega$. Then we calculate the density of each vertex in all observed quadruplet indices. We delete any vertex where the density is lower than 0.05. If that results in a viewing graph with too few cameras, the threshold is lowered to 0.02 and then 0.01. One of the datasets in ETH3D does not provide a dense enough graph and is discarded. To compare with the joint optimization framework and other global synchronization methods, we restrict our algorithm to the calibrated setting. We compare with the following methods: TrifocalSync [27], NRFM [4], MPLS [13], LUD [16], BATA [18], and a very recent SOTA method Cycle-Sync [19]. Note that BATA and Cycle-Sync are location synchronization algorithms. We use MPLS for their rotation synchronization component.

5.4.1. ETH3D

We apply our methods to 11 stereo multiview benchmark datasets in ETH3D from [48, 49]. ETH3D is a diverse collection of datasets, ranging from artificial to natural as well as indoor to outdoor environments. It has been used widely as benchmarks for SfM. We preprocess the datasets such that if the two view viewing graph contains weakly connected components, we retain only the largest component. We estimate the keypoint matches using SIFT feature points. We then estimate the trifocal tensors with the implementations in [50]. We construct the block quadrifocal tensor following the procedure in Section 5.2. We also estimate the essential matrices using RANSAC for comparison purposes.

5.4.2. EPFL

We also apply our methods to 6 multiview stereo high resolution datasets in EPFL [51]. We use GlueStick [33] to detect and match feature points. We then estimate the trifocal tensors following the same procedure as with ETH3D. We estimate the fundamental matrices with GC-RANSAC [52] for comparison purposes. We then form the estimated block quadrifocal tensor.

5.4.3. Results

The mean location errors for the ETH3D and EPFL datasets are displayed in Figures 1 and 2, respectively. We also report the median location errors, the mean rotation errors, and the median rotation errors, and refer to tables in the appendices for comprehensive results. In terms of location quality, we see that QuadSync and Joint Opt. perform the best or very close to the best in 7/11 of the datasets in ETH3D, and 4/6 of the EPFL datasets. Since our methods operate on higher-order measurements, they prefer dense viewing graphs. Among most of the datasets on which we perform poorly, the ratio of the number of estimated quadrifocal blocks to the total number of blocks drops below 30%. However, among the datasets where this ratio is larger than 70%, our proposed methods tend to find better quality solutions than SOTA algorithms. Having clean estimations and a high completion rate are important for the success of the algorithm. The completion rate and runtime of our algorithm can be found in the appendix.

Remark 6. *Since we operate on dense graphs which are computationally heavy, our methods may perform best with distributed synchronization approaches. Two small experiments to demonstrate the potential of applying the methods to large scale datasets, when given good clusters, is included in the appendices. We hope our work may motivate more research on distributed approaches for higher-order methods. As mentioned in Remark 1, another advantage of using quadrifocal tensors is that they become less sensitive in the case of collinear configurations. We experiment on a near-collinear subsequence of views from the plant_scene_1 dataset from ETH3D SLAM, see Figure 3 for the reconstructed camera poses. Note that this configuration can't be reconstructed with the pairwise measurements of fundamental matrices.*

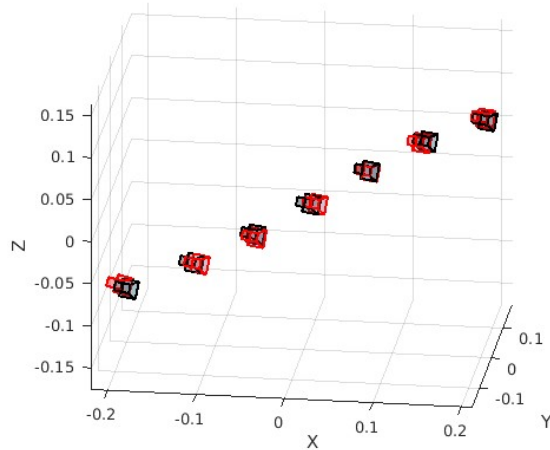


Figure 3. QuadSync retrieved camera poses on near-collinear views from plant_scene_1 dataset from ETH3D SLAM.

Another small synthetic experiment to show the effectiveness of our theory and algorithm for collinear configurations of cameras is included in the appendices. We see that the algorithm is insensitive to the collinear configuration and can still successfully synchronize the quadrifocal tensors.

6. Conclusion

We investigated totally novel approaches to synchronization in SfM. We introduced the block quadrifocal tensor, and laid theoretical foundations for its synchronization. We established an explicit Tucker decomposition for the block quadrifocal

tensor, in which the factor matrices are exactly the camera poses. We established the low multilinear rank of $(4, 4, 4, 4)$ and low projection rank of $(2, 2, 2, 2, 2, 2)$. We showed that the multilinear rank is sufficient to determine the scales for the block quadrifocal tensor, enabling camera pose retrieval. We also established additional new properties of the n -view essential matrix and the block trifocal tensor. We devised the algorithms QuadSync and Joint Opt. for the synchronization of the block quadrifocal tensor, and for the joint synchronization of all three entities together. Experiments showed the methods can improve accuracy in synchronization, especially location accuracy. In sum, we provided the first ever work on synchronization through quadrifocal tensors, and showed that there is great promise for advancing SfM systems via higher-order measurements.

There are many opportunities for future work. Though we develop strong theory, the algorithms depend on the quality of the block estimations. This motivates work on estimating and processing quadrifocal tensors. Our algorithms also require dense subsets and are more computationally demanding than two view and three view methods. The development of distributed synchronization methods for higher-order methods is needed.

Acknowledgement

D.M. and G.L. were supported in part by NSF award DMS 2152766. D.M. was also supported by the DSI-MnDRIVE PhD Graduate Assistantship from the University of Minnesota Data Science Initiative and Minnesota’s Discovery, Research, and Innovation Economy. J.K. was supported in part by NSF awards DMS 2309782 and CISE-IIS 2312746, the DOE award SC0025312, the Sloan Foundation, and start-up grants from the College of Natural Science and Oden Institute at the University of Texas at Austin.

We thank Shaohan Li and Michael Thomas for helpful discussions on processing ETH3D datasets. We thank Yifan Zhang for suggestions on tuning ADMM algorithms with tensors.

References

- [1] SriRam Thirthala and Marc Pollefeys. Radial multi-focal tensors: Applications to omnidirectional camera calibration. *IJCV*, 96(2):195–211, 2012. [1](#)
- [2] Noah Snavely, Steven M Seitz, and Richard Szeliski. Photo Tourism: Exploring photo collections in 3D. In *SIGGRAPH*, pages 835–846. 2006. [1](#)
- [3] Johannes L Schonberger and Jan-Michael Frahm. Structure-from-motion revisited. In *CVPR*, pages 4104–4113, 2016. [1](#)
- [4] Soumyadip Sengupta, Tal Amir, Meirav Galun, Tom Goldstein, David W Jacobs, Amit Singer, and Ronen Basri. A new rank constraint on multi-view fundamental matrices, and its application to camera location recovery. In *CVPR*, pages 4798–4806, 2017. [1](#), [2](#), [11](#)
- [5] Yoni Kasten, Amnon Geifman, Meirav Galun, and Ronen Basri. GPSfM: Global projective sfm using algebraic constraints on multi-view fundamental matrices. In *CVPR*, pages 3264–3272, 2019. [2](#)
- [6] Yoni Kasten, Amnon Geifman, Meirav Galun, and Ronen Basri. Algebraic characterization of essential matrices and their averaging in multiview settings. In *ICCV*, pages 5895–5903, 2019. [2](#)
- [7] Amnon Geifman, Yoni Kasten, Meirav Galun, and Ronen Basri. Averaging essential and fundamental matrices in collinear camera settings. In *CVPR*, pages 13–19, 2020. [1](#), [2](#), [4](#)
- [8] Rakshith Madhavan and Federica Arrigoni. On the recovery of cameras from fundamental matrices. In *ICCV*, pages 20934–20943, 2025. [1](#)
- [9] Venu Madhav Govindu. Lie-algebraic averaging for globally consistent motion estimation. In *CVPR*, volume 1, pages 1–8, 2004. [2](#)
- [10] Avishek Chatterjee and Venu Madhav Govindu. Efficient and robust large-scale rotation averaging. In *ICCV*, pages 521–528, 2013.
- [11] Richard Hartley, Jochen Trumpf, Yuchao Dai, and Hongdong Li. Rotation averaging. *IJCV*, 103(3):267–305, 2013.

- [12] Avishek Chatterjee and Venu Madhav Govindu. Robust relative rotation averaging. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(4):958–972, 2017.
- [13] Yunpeng Shi and Gilad Lerman. Message passing least squares framework and its application to rotation synchronization. In *ICML*, pages 8796–8806. PMLR, 2020. [11](#)
- [14] Mica Arie-Nachimson, Shahar Z Kovalsky, Ira Kemelmacher-Shlizerman, Amit Singer, and Ronen Basri. Global motion estimation from point matches. In *2012 Second International Conference on 3D Imaging, Modeling, Processing, Visualization & Transmission*, pages 81–88. IEEE, 2012. [2](#)
- [15] Kyle Wilson and Noah Snavely. Robust global translations with 1DSFM. In *ECCV*, pages 61–75. Springer, 2014. [2](#)
- [16] Onur Ozyesil, Amit Singer, and Ronen Basri. Stable camera motion estimation using convex programming. *SIAM Journal on Imaging Sciences*, 8(2):1220–1262, 2015. [11](#)
- [17] Thomas Goldstein, Paul Hand, Choongbum Lee, Vladislav Voroninski, and Stefano Soatto. Shapefit and Shapekick for robust, scalable structure from motion. In *ECCV*, pages 289–304. Springer, 2016.
- [18] Bingbing Zhuang, Loong-Fah Cheong, and Gim Hee Lee. Baseline desensitizing in translation averaging. In *CVPR*, pages 4539–4547, 2018. [11](#)
- [19] Shaohan Li, Yunpeng Shi, and Gilad Lerman. Cycle-Sync: Robust global camera pose estimation through enhanced cycle-consistent synchronization. *arXiv preprint arXiv:2511.02329*, 2025. [2](#), [11](#)
- [20] Federica Arrigoni, Beatrice Rossi, and Andrea Fusiello. Spectral synchronization of multiple views in SE (3). *SIAM Journal on Imaging Sciences*, 9(4):1963–1990, 2016. [2](#)
- [21] David M Rosen, Luca Carlone, Afonso S Bandeira, and John J Leonard. SE-Sync: A certifiably correct algorithm for synchronization over the special Euclidean group. *The International Journal of Robotics Research*, 38(2-3):95–125, 2019.
- [22] Mihai Cucuringu, Yaron Lipman, and Amit Singer. Sensor network localization by eigenvector synchronization over the Euclidean group. *ACM Transactions on Sensor Networks (TOSN)*, 8(3):1–42, 2012.
- [23] Jesus Briales and Javier Gonzalez-Jimenez. Cartan-Sync: Fast and global SE(d)-synchronization. *IEEE Robotics and Automation Letters*, 2(4):2127–2134, 2017. [2](#)
- [24] Linfei Pan, Dániel Baráth, Marc Pollefeys, and Johannes L Schönberger. Global structure-from-motion revisited. In *ECCV*, pages 58–77. Springer, 2024. [2](#)
- [25] Christopher Sweeney, Tobias Hollerer, and Matthew Turk. Theia: A fast and scalable structure-from-motion library. In *ACMMM*, pages 693–696, 2015. [2](#)
- [26] Shaohan Li, Yunpeng Shi, and Gilad Lerman. Efficient detection of long consistent cycles and its application to distributed synchronization. In *CVPR*, pages 5260–5269, 2024. [2](#)
- [27] Daniel Miao, Gilad Lerman, and Joe Kileel. Tensor-based synchronization and the low-rankness of the block trifocal tensor. In *NeurIPS*, volume 37, pages 69505–69532, 2024. [3](#), [10](#), [11](#), [16](#)
- [28] Adriana L Duncan and Joe Kileel. Higher-order group synchronization. *arXiv preprint arXiv:2505.21932*, 2025. [2](#), [9](#)
- [29] Petr Hruby, Viktor Korotynskiy, Timothy Duff, Luke Oeding, Marc Pollefeys, Tomas Pajdla, and Viktor Larsson. Four-view geometry with unknown radial distortion. In *CVPR*, pages 8990–9000, 2023. [2](#)
- [30] Andrew I Comport, Ezio Malis, and Patrick Rives. Accurate quadrifocal tracking for robust 3D visual odometry. In *Proceedings 2007 IEEE International Conference on Robotics and Automation*, pages 40–45. IEEE, 2007. [2](#)
- [31] Andrew I Comport, Ezio Malis, and Patrick Rives. Real-time quadrifocal visual odometry. *The International Journal of Robotics Research*, 29(2-3):245–266, 2010. [2](#)
- [32] Shaohui Liu, Yifan Yu, Rémi Pautrat, Marc Pollefeys, and Viktor Larsson. 3D line mapping revisited. In *CVPR*, pages 21445–21455, 2023. [3](#)
- [33] Rémi Pautrat, Iago Suárez, Yifan Yu, Marc Pollefeys, and Viktor Larsson. GlueStick: Robust image matching by sticking points and lines together. In *ICCV*, pages 9706–9716, 2023. [3](#), [12](#)
- [34] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge university press, 2003. [3](#), [11](#), [16](#), [18](#)

- [35] Richard I Hartley. Computation of the quadrifocal tensor. In *ECCV*, pages 20–35. Springer, 1998.
- [36] Luke Oeding. The quadrifocal variety. *Linear Algebra and its Applications*, 512:306–330, 2017. [3](#)
- [37] Amnon Shashua and Lior Wolf. On the structure and properties of the quadrifocal tensor. In *ECCV*, pages 710–724, 2000. [3](#)
- [38] Ali Elqursh and Ahmed Elgammal. Line-based relative pose estimation. In *CVPR*, pages 3049–3056, 2011. [3](#)
- [39] Yubin Kuang and Kalle Astrom. Pose estimation with unknown focal length using points, directions and lines. In *ICCV*, pages 529–536, 2013.
- [40] Zuzana Kukelova, Joe Kileel, Bernd Sturmfels, and Tomas Pajdla. A clever elimination strategy for efficient minimal solvers. In *CVPR*, pages 4912–4921, 2017.
- [41] Pedro Miraldo, Tiago Dias, and Srikumar Ramalingam. A minimal closed-form solution for multi-perspective pose estimation using points and lines. In *ECCV*, pages 474–490, 2018.
- [42] Joe Kileel, Zuzana Kukelova, Tomas Pajdla, and Bernd Sturmfels. Distortion varieties. *Foundations of Computational Mathematics*, 18(4):1043–1071, 2018. [3](#)
- [43] Joe Kileel. Minimal problems for the calibrated trifocal variety. *SIAM Journal on Applied Algebra and Geometry*, 1(1):575–598, 2017.
- [44] Chris Aholt and Luke Oeding. The ideal of the trifocal variety. *Mathematics of Computation*, 83(289):2553–2574, 2014. [5](#)
- [45] Joe Kileel and Kathlen Kohn. Snapshot of algebraic vision. *AMS Proceedings of Symposia in Pure Mathematics*, 111:323–363, 2025. [3](#)
- [46] Tamara G Kolda and Brett W Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009. [4](#)
- [47] Feng Yu, Teng Zhang, and Gilad Lerman. A subspace-constrained Tyler’s estimator and its applications to structure from motion. In *CVPR*, pages 14575–14584, 2024. [11](#)
- [48] Thomas Schöps, Johannes L. Schönberger, Silvano Galliani, Torsten Sattler, Konrad Schindler, Marc Pollefeys, and Andreas Geiger. A multi-view stereo benchmark with high-resolution images and multi-camera videos. In *CVPR*, pages 3260–3269, 2017. [11](#)
- [49] Thomas Schöps, Torsten Sattler, and Marc Pollefeys. BAD SLAM: Bundle adjusted direct RGB-D SLAM. In *CVPR*, pages 134–144, 2019. [11](#)
- [50] Laura F Julià and Pascal Monasse. A critical review of the trifocal tensor estimation. In *Pacific-Rim Symposium on Image and Video Technology*, pages 337–349. Springer, 2017. [11](#)
- [51] Christoph Strecha, Wolfgang Von Hansen, Luc Van Gool, Pascal Fua, and Ulrich Thoennessen. On benchmarking camera calibration and multi-view stereo for high resolution imagery. In *CVPR*, pages 1–8, 2008. [12](#)
- [52] Daniel Barath and Jiří Matas. Graph-cut RANSAC. In *CVPR*, pages 6733–6741, 2018. [12](#)
- [53] David S. Dummit and Richard M. Foote. *Abstract Algebra*. John Wiley & Sons, 3rd edition, 2004. [16](#)
- [54] Roberto Tron, Luca Carlone, Frank Dellaert, and Kostas Daniilidis. Rigid components identification and rigidity control in bearing-only localization using the graph cycle basis. In *2015 American Control Conference (ACC)*, pages 3911–3918. IEEE, 2015. [23](#)
- [55] Federica Arrigoni, Andrea Fusiello, and Beatrice Rossi. Camera motion from group synchronization. In *2016 Fourth International Conference on 3D Vision (3DV)*, pages 546–555. IEEE, 2016. [23](#)

Appendices

7. Additional Background Material

7.1. Line Projection Matrices

Let $A \in \mathbb{R}^{3 \times 4}$ be a projection matrix, factoring as $KR[I|t]$ where $R \in SO(3)$ and $t \in \mathbb{R}^3$. Then, the exterior square is

$$\mathcal{P} = \begin{bmatrix} A^2 \wedge A^3 \\ A^3 \wedge A^1 \\ A^1 \wedge A^2 \end{bmatrix}$$

where A^i indicates the i th row of the camera matrix A . Here, $\wedge$ is the wedge product, or the exterior product, between two vectors. The order of the basis is given by the order in the *columnindices* in the following explanation. This can be calculated specifically in the following way. First, let *rowindices* = [2,3; 1,3; 1,2], *columnindices* = [1,2; 1,3; 1,4; 2,3; 2,4; 3,4]. Then let $\mathcal{P}$ be the 3×6 matrix, where $\mathcal{P}_{i,j}$ is calculated by the minors

$$\det(A(\text{rowindices}(i,:), \text{columnindices}(j,:)))$$

if $i=1,3$, and by

$$-\det(A(\text{rowindices}(i,:), \text{columnindices}(j,:)))$$

if $i=2$. When A is a $3n \times 4$ matrix, then let $\mathcal{P}$ denote the $3n \times 6$ matrix of the stacked exterior squares. We refer to [53] for a more detailed description of the exterior algebra of a vector space. Note that the matrix $\mathcal{P}$ is also the line projection matrices, which can project 3D world lines in plucker coordinates (which are in $\mathbb{R}^6$) to lines in the image plane. We refer to [34] for more details on the role of line projection matrices in computer vision.

8. Proofs for Theorems

8.1. Block Trifocal Tensor in Collinear Cases

The reference [27] includes a characterization of the low multilinear rank for the block trifocal tensor under the non-collinear setting. We extend this to the collinear setting, showing that the block trifocal tensor will then have a low multilinear rank of (5,4,4).

Theorem 8.1. *Given a block trifocal tensor with appropriately chosen blockwise scales. If the n cameras that produce $\mathcal{T}^n$ are all collinear and do not all share the same camera center, then $\text{mlrank}(\mathcal{T}^n) = (5,4,4)$.*

Proof. Since it has been shown in the proof for Theorem 3.1 that the stacked camera matrices will only exhibit a low rank if all cameras lie at the same point, it is the same case for the low ranks in the flattenings of the second and third mode. We only need to show the rank drop in the first mode. Note that $\mathcal{T}_{(1)}^n = \mathcal{P}(\mathcal{G}_T)_{(1)}(C \otimes C)^T$, we just need to show that $\mathcal{P} \in \mathbb{R}^{3n \times 6}$ has rank 5. Since for individual line projection matrices $\mathcal{P}_i$, we have $\mathcal{P}_i \mathcal{L} = 0$ only when $\mathcal{L}$ is a line that passes through the camera center. Since we have a collinear set of cameras, there is only one line that passes through all of the camera centers, hence the kernel of $\mathcal{P}$ has dimension 1. $\square$

8.2. Proof for Theorem 3.1

Proof. By expanding equation (1) and using the definition of the Tucker product

$$(\mathcal{G}_Q \times_1 C \times_2 C \times_3 C \times_4 C)_{pqrs} = \sum_{a=1}^4 \sum_{b=1}^4 \sum_{c=1}^4 \sum_{d=1}^4 (\mathcal{G}_Q)_{abcd} C_{ap} C_{bq} C_{cr} C_{ds}, \quad (12)$$

we can see that the core $\mathcal{G}_Q$ admits the following structure:

$$(\mathcal{G}_Q)_{abcd} = \begin{cases} \text{sgn}(abcd) & \text{if } a, b, c, d \in \{1, 2, 3, 4\} \text{ are not all the same} \\ 0 & \text{otherwise.} \end{cases}$$

Then, each quadrifocal tensor block $\mathcal{Q}_{ijkl}^n$ calculated through the Tucker decomposition will be the exact same as the quadrifocal tensor calculated using (1). Now, we establish the full rankness of C in the non-collinear case. Suppose that $\text{rank}(C) < 4$, there exists $x \in \mathbb{R}^4$ such that $Cx = 0$. This is equivalent to $P_i x = 0$ for any $i = 1, \dots, n$. However, $P_i x = 0$ only when x is the camera centre of the camera. This means that x is the camera center for all cameras, which is a contradiction. Thus, if the cameras that produce $\mathcal{Q}^n$ don't share the same camera center, the multilinear rank of $\mathcal{Q}^n = (4, 4, 4, 4)$, even in the collinear case. $\square$

8.3. Proof for Theorem 3.2

Proof. We prove by explicitly calculating the contractions with two vectors. We know $\mathcal{Q}^n$ admits the factorization $\mathcal{Q}^n = \mathcal{G}_Q \times_1 C \times_2 C \times_3 C \times_4 C$, where $\mathcal{G}_Q \in \mathbb{R}^{4 \times 4 \times 4 \times 4}$. Denote $\text{P-Rank}(T)$ as $(M_1, M_2, M_3, M_4, M_5, M_6)$. Calculating the P-Rank involves taking generic linear combinations of the slices of the tensor, which is equivalent to contracting with a matrix $X \in \mathbb{R}^{3n \times 3n}$. For example, for M_1 it can be calculated as the rank for $\mathcal{Q}_{p12}^n = \sum_{i=1}^{3n} \sum_{j=1}^{3n} X_{ij} \mathcal{Q}_{ij::}^n$.

(1) (M_1) Recall that

$$\mathcal{Q}_{ijkl}^n = (\mathcal{G}_Q \times_1 C \times_2 C \times_3 C \times_4 C)_{ijkl} = \sum_{a=1}^4 \sum_{b=1}^4 \sum_{c=1}^4 \sum_{d=1}^4 \mathcal{G}_{abcd} C_{ia} C_{jb} C_{kc} C_{ld}.$$

Then

$$\left(\sum_{i,j} X_{ij} \mathcal{Q}_{ij::}^n \right)_{kl} = \sum_{i,j} X_{ij} \sum_{a=1}^4 \sum_{b=1}^4 \sum_{c=1}^4 \sum_{d=1}^4 \mathcal{G}_{abcd} C_{ia} C_{jb} C_{kc} C_{ld} = \sum_{a=1}^4 \sum_{b=1}^4 \sum_{c=1}^4 \sum_{d=1}^4 \mathcal{G}_{abcd} \left(\sum_{i,j} X_{ij} C_{ia} C_{jb} \right) C_{kc} C_{ld}.$$

Let $S_{ab} = \sum_{i,j} X_{ij} C_{ia} C_{jb}$, or in other words $S = C^T X C$. Then, we have

$$\begin{aligned} \left(\sum_{i,j} X_{ij} \mathcal{Q}_{ij::}^n \right)_{kl} &= \sum_{a=1}^4 \sum_{b=1}^4 \sum_{c=1}^4 \sum_{d=1}^4 \mathcal{G}_{abcd} S_{ab} C_{kc} C_{ld} \\ &= \sum_{c=1}^4 \sum_{d=1}^4 \left(\sum_{a=1}^4 \sum_{b=1}^4 \mathcal{G}_{abcd} S_{ab} \right) C_{kc} C_{ld} = \left(C \left(\sum_{a=1}^4 \sum_{b=1}^4 \mathcal{G}_{abcd} S_{ab} \right) C^T \right)_{kl}. \end{aligned}$$

Here, $\mathcal{G}_S \in \mathbb{R}^{4 \times 4}$ is given by

$$\mathcal{G}_S = \sum_{a=1}^4 \sum_{b=1}^4 \mathcal{G}_{abcd} S_{ab} = \begin{bmatrix} 0 & S_{34} - S_{43} & S_{42} - S_{24} & S_{23} - S_{32} \\ S_{43} - S_{34} & 0 & S_{14} - S_{41} & S_{31} - S_{13} \\ S_{24} - S_{42} & S_{41} - S_{14} & 0 & S_{12} - S_{21} \\ S_{32} - S_{23} & S_{13} - S_{31} & S_{21} - S_{12} & 0 \end{bmatrix}$$

By our definition of projection rank, $\text{rank}(X) = 1$, so $X = xy^T$ for some generic $x, y \in \mathbb{R}^4$. In general, we can explicitly calculate the rank of $\mathcal{G}_S$, where the reduced row echelon form is

$$\text{rref}(\mathcal{G}_S) = \begin{bmatrix} 1 & 0 & -(x_1 y_4 - x_4 y_1)/(x_3 y_4 - x_4 y_3) & -(x_1 y_3 - x_3 y_1)/(x_3 y_4 - x_4 y_3) \\ 0 & 1 & -(x_2 y_4 - x_4 y_2)/(x_3 y_4 - x_4 y_3) & -(x_2 y_3 - x_3 y_2)/(x_3 y_4 - x_4 y_3) \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

when $x_3y_4 - x_4y_3 \neq 0$. Thus, for generic X , $\text{rank}(\mathcal{G}_S) = 2$, so that $\text{rank}(C\mathcal{G}_SC^T) = 2$, proving the projection rank for M_1 to be 2. This has also been checked numerically and symbolically.

- (2) M_2, M_3, M_4, M_5, M_6 : Due to symmetries, $M_2, \dots, M_6$ will have the same ranks and properties. We omit the details. This establishes Theorem 3.2. $\square$

8.4. Proof for Proposition 3.3

Proof. These are mainly facts from [34], but we still describe the rationale and how it corresponds to the descriptions in the reference. Recall that the quadrifocal tensor can be calculated via the following formula (1) given four cameras P_i, P_j, P_k, P_l :

$$(\mathcal{Q}_{ijkl}^n)_{pqrs} = \det \begin{bmatrix} P_i^p \\ P_j^q \\ P_k^r \\ P_l^s \end{bmatrix}.$$

- (1) For $\mathcal{Q}_{iiii}^n$ diagonal blocks, all four cameras will be the same in the definition for quadrifocal tensors from camera matrices. Since P_i only has 3 rows, there must be a repeating row in the determinant, meaning that $\mathcal{Q}_{iiii}^n = 0$ in $\mathbb{R}^{3 \times 3 \times 3 \times 3}$.
- (2) Suppose that we now have 3 overlapping indices, such that we are considering the blocks $\mathcal{Q}_{iiij}^n, \mathcal{Q}_{iiji}^n, \mathcal{Q}_{ijii}^n, \mathcal{Q}_{jiii}^n$ for $i \neq j$. Then, among the four rows in the determinant, three must correspond to the same camera i . By [34], these entries will be the epipoles up to signs, where epipoles are the images of the camera center of view i in view j .
- (3) Suppose that we now have 2 overlapping indices, such that we are looking at the blocks $\mathcal{Q}_{iijk}^n, \mathcal{Q}_{ijik}^n, \mathcal{Q}_{ijk i}^n, \mathcal{Q}_{jiik}^n, \mathcal{Q}_{jik i}^n, \mathcal{Q}_{jkii}^n, \mathcal{Q}_{ikij}^n, \mathcal{Q}_{ikji}^n, \mathcal{Q}_{kij i}^n, \mathcal{Q}_{kjii}^n$ for $i \neq j, i \neq k, j \neq k$. Then, among the four rows in the determinant, two correspond to the same camera i . Again by [34], these entries will be the elements of the trifocal tensor up to signs.
- (4) Suppose that we now have 2 overlapping indices, and the other two are also the same, such that we are looking at the blocks, $\mathcal{Q}_{iijj}^n, \mathcal{Q}_{ijij}^n, \mathcal{Q}_{ijji}^n, \mathcal{Q}_{jii j}^n, \mathcal{Q}_{jiji}^n, \mathcal{Q}_{jjii}^n$ for $i \neq j$. Then, among the four rows in the determinant, two correspond to the same camera i , and the other two correspond to the same camera j . Again by [34], these entries will be the elements of the fundamental matrices up to signs.

This completes the proof. $\square$

8.5. Proof for Theorem 3.4

Proof. Blockwise multiplication by a rank-1 tensor with non-vanishing entries preserves multilinear rank, as this operation is equivalent to a Tucker product with invertible diagonal matrices.

Hence, without loss of generality, assume $\lambda_{i111} = \lambda_{1j11} = \lambda_{11k1} = \lambda_{111\ell} = 1$ for all $i, j, k, \ell \in \{2, \dots, n\}$. We will show below that for some $c \in \mathbb{R}^*$, the entries satisfy:

- $\lambda_{ijk\ell} = c$ if exactly two of i, j, k, ℓ equal 1;
- $\lambda_{ijk\ell} = c^2$ if exactly one of i, j, k equals 1;
- $\lambda_{ijk\ell} = c^3$ if none of i, j, k, ℓ equal 1 and the indices are not all the same.

This result will establish the theorem, since setting $\alpha = \beta = \gamma = (1, c, \dots, c)$ and $\delta = (\frac{1}{c}, 1, \dots, 1)$ ensures $\lambda_{ijk\ell} = \alpha_i \beta_j \gamma_k \delta_\ell$ whenever i, j, k, ℓ are not all equal.

We let $\mathcal{Q}_{(1)}^n$ and $(\lambda \odot_b \mathcal{Q}^n)_{(1)}$ denote the mode-1 matrix flattenings in $\mathbb{R}^{3n \times 27n^3}$ of the block quadrifocal tensor and its scaled variant, where rows correspond to the first mode of the tensors. Invoking Theorem 3.1 alongside our standing assumptions shows that both matrices have rank 4. Consequently, every 5×5 minor of these matrices must be zero.

We will exploit this by analyzing specific 5×5 submatrices of $(\lambda \odot_b \mathcal{Q}^n)_{(1)}$ to construct a system of constraints on λ , ultimately establishing the existence of the constant c . We adopt the index convention (ip) for rows and $(jq, kr, \ell s)$ for columns, where $i, j, k, \ell \in [n]$ and $p, q, r, s \in [3]$. Thus, entries satisfy $((\lambda \odot_b \mathcal{Q}^n)_{(1)})_{(ip), (jq, kr, \ell s)} = \lambda_{ijk\ell} (\mathcal{Q}_{ijk\ell}^n)_{pqrs}$.

Case 1: The first submatrix of $(\lambda \odot_b \mathcal{Q}^n)_{(1)}$ we consider has column indices $(i1, 13, 12), (12, j2, 11), (12, j3, 11),$

$(13,j3,12)$, $(11,j1,13)$ and row indices (11) , (12) , (13) , $(i1)$, $(i2)$, where $i,j \in \{2, \dots, n\}$. Explicitly, the submatrix is

$$\begin{bmatrix} (\mathcal{Q}_{i11}^n)_{1132} & (\mathcal{Q}_{i1j1}^n)_{1221} & (\mathcal{Q}_{i1j1}^n)_{1231} & (\mathcal{Q}_{i1j1}^n)_{1332} & (\mathcal{Q}_{i1j1}^n)_{1113} \\ (\mathcal{Q}_{i11}^n)_{2132} & (\mathcal{Q}_{i1j1}^n)_{2221} & (\mathcal{Q}_{i1j1}^n)_{2231} & (\mathcal{Q}_{i1j1}^n)_{2332} & (\mathcal{Q}_{i1j1}^n)_{2113} \\ (\mathcal{Q}_{i11}^n)_{3132} & (\mathcal{Q}_{i1j1}^n)_{3221} & (\mathcal{Q}_{i1j1}^n)_{3231} & (\mathcal{Q}_{i1j1}^n)_{3332} & (\mathcal{Q}_{i1j1}^n)_{3113} \\ \lambda_{ii11}(\mathcal{Q}_{i11}^n)_{1132} & \lambda_{i1j1}(\mathcal{Q}_{i1j1}^n)_{1221} & \lambda_{i1j1}(\mathcal{Q}_{i1j1}^n)_{1231} & \lambda_{i1j1}(\mathcal{Q}_{i1j1}^n)_{1332} & \lambda_{i1j1}(\mathcal{Q}_{i1j1}^n)_{1113} \\ \lambda_{ii11}(\mathcal{Q}_{i11}^n)_{2132} & \lambda_{i1j1}(\mathcal{Q}_{i1j1}^n)_{2221} & \lambda_{i1j1}(\mathcal{Q}_{i1j1}^n)_{2231} & \lambda_{i1j1}(\mathcal{Q}_{i1j1}^n)_{2332} & \lambda_{i1j1}(\mathcal{Q}_{i1j1}^n)_{2113} \end{bmatrix},$$

which we abbreviate as

$$\begin{bmatrix} * & * & * & * & * \\ * & * & * & * & * \\ * & * & * & * & * \\ \lambda_{ii11}* & \lambda_{i1j1}* & \lambda_{i1j1}* & \lambda_{i1j1}* & \lambda_{i1j1}* \\ \lambda_{ii11}* & \lambda_{i1j1}* & \lambda_{i1j1}* & \lambda_{i1j1}* & \lambda_{i1j1}* \end{bmatrix}, \quad (13)$$

with asterisk denoting the corresponding entry in $\mathcal{Q}_{(1)}^n$. We view the determinant of (13) as a polynomial with respect to λ . It has degree ≤ 2 in the variables $\lambda_{ii11}, \lambda_{i1j1}$. Observe that if $\lambda_{i1j1} = 0$, the bottom two rows of the matrix are linearly independent. Also if $\lambda_{i1j1} - \lambda_{ii11} = 0$, then (13) equals a 5×5 submatrix of $\mathcal{Q}_{(1)}^n$ with rows operations performed; therefore (13) is rank-deficient. It follows that the determinant of (13) takes the form

$$s\lambda_{i1j1}(\lambda_{i1j1} - \lambda_{ii11}).$$

Here the scale $s = s(P_1, P_i, P_j)$ is a polynomial in the camera matrices. Due to polynomiality, s is nonzero Zariski-generically if we can exhibit a *single* instance of matrices P_1, P_i, P_j where the determinant of (13) does not vanish identically for all $\lambda_{i1j1}, \lambda_{ii11}$. Furthermore, we just need an instance with $i=j$, as this corresponds to a specialization of the case $i \neq j$. Computational verification with a random numerical instance of P_1, P_i proves the non-vanishing. Recalling the standing assumptions, we deduce $\lambda_{i1j1} = \lambda_{ii11}$.

We apply the same argument to modewise permutations of $\lambda \odot_b \mathcal{Q}^n$ and $\mathcal{Q}^n$, and obtain

$$\lambda_{\pi(i1j1)} = \lambda_{\pi(ii11)} \quad \text{for all } i, j \in \{2, \dots, n\} \text{ and permutations } \pi.$$

The argument goes through as $\pi \cdot \mathcal{Q}^n$ and $\pi \cdot (\lambda \odot_b \mathcal{Q}^n)$ have multilinear ranks bounded by $(4, 4, 4, 4)$ and from equation 4. So (13) looks the same but with indices permuted and possibly a sign flip.

We now see that λ -entries with two 1-indices agree. Indeed, taking $i=j$ above gives $\lambda_{\pi_1(i1i1)} = \lambda_{\pi_2(ii11)}$ for all π_1 and π_2 that fix $(ii11)$ and $(i1i1)$ respectively. So $\lambda_{ii11} = \lambda_{\pi(ii11)}$ for all π . Taking $i \neq j$ gives $\lambda_{ii11} = \lambda_{\pi(i1j1)} = \lambda_{jj11}$ for all π . Together, there exists $c \in \mathbb{R}^*$ such that $c = \lambda_{\pi(ij11)}$ for all $i, j \in \{2, \dots, n\}$ and permutations π .

Case 2: Next we consider the submatrix of $(\lambda \odot_b \mathcal{Q}^n)_{(1)}$ with column indices $(j1, k3, 12)$, $(12, j2, 11)$, $(12, j3, 11)$, $(13, j3, 12)$, $(11, j1, 13)$ and row indices (11) , (12) , (13) , $(i1)$, $(i2)$, where $i, j, k \in \{2, \dots, n\}$. It looks like

$$\begin{bmatrix} c* & * & * & * & * \\ c* & * & * & * & * \\ c* & * & * & * & * \\ \lambda_{ijk1}* & c* & c* & c* & c* \\ \lambda_{ijk1}* & c* & c* & c* & c* \end{bmatrix}, \quad (14)$$

where asterisks denote corresponding entries in $\mathcal{Q}_{(1)}^n$. As a polynomial in c and λ_{ijk1} , the determinant of (14) is a scalar multiple of $c(c^2 - \lambda_{ijk1})$. This is because the polynomial has degree ≤ 3 , if $c=0$ then the bottom two rows of (14) are linearly dependent, and if $c^2 = \lambda_{ijk1}$ then (14) is a 5×5 submatrix of $\mathcal{Q}_{(1)}^n$ with row and column operations performed. The scale is a polynomial in P_1, P_i, P_j, P_k . It is Zariski-generically nonzero if we exhibit one instance of camera matrices

such that the determinant of (13) does not vanish for all c, λ_{ijk1} . Further, it suffices to find an instance where $i = j = k$, as all other cases specialize to this. Computational verification with a random numerical instance of P_1, P_i proves the non-vanishing. It follows that $c^2 = \lambda_{ijk1}$. Appealing to symmetry like before, $c^2 = \lambda_{\pi(ijk1)}$ for all $i, j, k \in \{2, \dots, n\}$ and permutations π . Summarizing, all λ -entries with a single 1-index equal c^2 .

Case 3: Consider the submatrix of $(\lambda \odot \mathcal{Q}^n)_{(1)}$ with columns $(j1, k3, \ell 2)$, $(12, i2, 11)$, $(12, i3, 11)$, $(13, i3, 12)$, $(11, i1, 13)$ and rows (11) , (12) , (13) , $(i1)$, $(i2)$, where $i, j, k, \ell \in \{2, \dots, n\}$ and i, ℓ are distinct. The submatrix looks like

$$\begin{bmatrix} c^2* & * & * & * & * \\ c^2* & * & * & * & * \\ c^2* & * & * & * & * \\ \lambda_{ijk\ell}* & c* & c* & c* & c* \\ \lambda_{ijk\ell}* & c* & c* & c* & c* \end{bmatrix}. \quad (15)$$

The determinant of (15) is $c(c^3 - \lambda_{ijk\ell})$ multiplied by a polynomial in $P_1, P_i, P_j, P_k, P_\ell$. The most specialized case is $i = j = k$. Computer verification with a random numerical instance proves the polynomial is not identically zero. We deduce that $c^3 = \lambda_{ijk\ell}$. By symmetry, $c^3 = \lambda_{\pi(ijk\ell)}$ for all $i, j, k, \ell \in \{2, \dots, n\}$ with i, ℓ distinct and all permutations π . In other words, λ -entries with no 1-indices and non-identical indices equal c^3 .

Putting cases 1, 2 and 3 together shows that λ takes the announced form. This proves Theorem 3.4. $\square$

8.6. Proof for Theorem 4.1

Proof. The proof of this theorem follows from explicit calculation. We just need to verify that

$$\mathcal{E}_{ij}^n = \mathcal{P}_i \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \mathcal{P}_j^T.$$

For each entry $(\mathcal{E}_{ij}^n)_{kl}$ in $\mathcal{E}_{ij}^n$,

$$(\mathcal{E}_{ij}^n)_{kl} = (-1)^{k+l} \det \begin{bmatrix} \sim P_i^l \\ \sim P_j^k \end{bmatrix} = (\mathcal{P}_i)_k \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} (\mathcal{P}_j)_l^T$$

by expanding every entry. A direct expansion shows the result. This has also been checked numerically. $\square$

8.7. Proof for Theorem 4.2

Proof. We prove this by explicit calculation. The block tensor $\mathcal{T}^n$ admits the factorization $\mathcal{T}^n = \mathcal{G} \times_1 \mathcal{P} \times_2 C \times_3 C$, where $\mathcal{G} \in \mathbb{R}^{6 \times 4 \times 4}$. Denote P-Rank($\mathcal{T}^n$) as (M_1, M_2, M_3) . Calculating the P-Rank involves taking generic linear combinations of the slices of the tensor, this is equivalent to contracting with a vector. In the tensor-matrix product, let $x \in \mathbb{R}^{3n \times 1}$, $\sum_{i=1}^{3n} x_i \mathcal{T}_{ijk}^n = \mathcal{G} \times_1 (x^T P) \times_2 C \times_3 C$. Then we explicitly calculate M_1 from $\mathcal{G} \times_1 (x^T P)$. The same process holds for modes 2 and 3 to compute M_2 and M_3 in the P-Rank.

(1) $\mathbf{M}_1$: Let $y = x^T P$. We have

$$\mathcal{T}_1^n = \sum_{i=1}^4 y_i \mathcal{G}_{ijk} = \begin{bmatrix} 0 & y_6 & -y_5 & y_4 \\ -y_6 & 0 & y_3 & -y_2 \\ -y_5 & -y_3 & 0 & y_1 \\ -y_4 & y_2 & -y_1 & 0 \end{bmatrix}.$$

Generically, this matrix $\mathcal{T}_1^n$ will clearly have rank 4.

(2) $\mathbf{M}_2$: Let $y = x^T C$, then we have

$$\mathcal{T}_2^n = \sum_{j=1}^4 y_j \mathcal{G}_{ijk} = \begin{bmatrix} 0 & 0 & 0 & y_4 & -y_3 & y_2 \\ 0 & -y_4 & y_3 & 0 & 0 & -y_1 \\ y_4 & 0 & -y_2 & 0 & y_1 & 0 \\ -y_3 & y_2 & 0 & -y_1 & 0 & 0 \end{bmatrix}$$

This matrix generically will clearly have rank 3. Denote R_i as the i -th row of $\mathcal{T}_2^n$. We can observe that the fourth row is a linear combination of rows 1,2,3, where $y_4 R_4 = y_1 R_1 + y_2 R_2 + y_3 R_3$. However, R_1, R_2, R_3 must be linearly independent in general.

(3) $\mathbf{M}_3$: This is the same as M_2 due to symmetry.

□

9. Algorithm Details

9.1. QuadSync

We present pseudocode for QuadSync in Algorithm 1 in this section.

Algorithm 1 Quadsync IRLS-ADMM

Input: $\mathcal{Q}^n \in \mathbb{R}^{3n \times 3n \times 3n \times 3n}$, $\rho > 0 \in \mathbb{R}$

Ω set of observed block indices

Output: $\bar{C} \in \mathbb{R}^{3n \times 4}$

Normalize $\mathcal{Q}^n$ so that each block has norm 1

Obtain C_i, B from first four singular vectors in first factor matrix of HOSVD($\mathcal{Q}^n$)

Calculate initial IRLS weights W_Q from (5)

while not converged **do**

Now we run the ADMM cases

while not converged **do**

First optimize for C_i, Λ alternatingly

while not converged **do**

 Update $C_i, i = 1, 2, 3, 4$ via (6)

 Update Λ_Q via (7)

end while

 Update B via (8)

 Update $\Gamma_i, i = 1, 2, 3, 4$ via (9)

end while

 Update new IRLS weights W_Q via (5)

end while

$\bar{C} = \text{avg}(C_1, C_2, C_3, C_4)$

Return

We count the storage and flop complexities of QuadSync. Let n be the number of cameras. Then the input of QuadSync consists of a block quadrifocal tensor with up to $81n^4$ elements. The auxiliary variables consist of $C_1, C_2, C_3, C_4, \Lambda, W, B, \Gamma_1, \Gamma_2, \Gamma_3$, with size $12n \times 8 + 2|\Omega| = O(n^4)$. Thus, the overall storage complexity is $O(n^4)$.

For the flop complexity, we first account for the initialization. For the initialization, C is calculated via HOSVD of a matrix of size $3n \times 27n^3$ where the target rank is 4. This costs $O(4 \times 81 \times n^4) = O(n^4)$ flops. For the running iterations, to calculate C , one has to multiply a matrix of size $4 \times 27n^3$ with a diagonal matrix of size $27n^3 \times 27n^3$, then take the product and multiply it with another matrix of size $27n^3 \times 4$. This multiplication of three matrix will have to be done for a total of n times for each update of the stacked camera matrices C . Thus, the flop complexity for calculating the updates for C is $O(n^4)$. Calculating Λ_Q, W also requires $O(|\Omega|) = O(n^4)$ flops. Calculating B, Γ_i requires $O(n)$ flops. The overall flop complexity is $O(n^4)$.

9.2. Joint Opt.

Though similar to the above, we include detailed calculations for the steps of Joint Opt. and include pseudocode in Algorithm 2. We first give explicit formulas for all the updates that we use.

- (1) $\mathbf{W}_Q, \mathbf{W}_T, \mathbf{W}_E$: Here W is the set of IRLS weights integrated in the norm, so

$$w_{ijkl} = \begin{cases} 1/m_{ijkl}^t & \text{if } (i,j,k,l) \in \Omega \\ 0 & \text{otherwise.} \end{cases}$$

For W_Q we have

$$(m_Q)_{ijkl}^t = \max(\delta, \text{sqrt}(\|(\Lambda_Q)^{(t-1)} \tilde{\mathcal{Q}}_{ijkl}^n - \mathcal{G}_Q \times_1 (C_1)_i^{(t-1)} \times_2 (C_2)_j^{(t-1)} \times_3 (C_3)_k^{(t-1)} \times_4 (C_4)_l^{(t-1)}\|_F)). \quad (16)$$

For W_T we have

$$(m_T)_{ijkl}^t = \max(\delta, \text{sqrt}(\|(\Lambda_T)^{(t-1)} \tilde{\mathcal{T}}_{ijkl}^n - \mathcal{G}_T \times_1 (P_1)_i^{(t-1)} \times_2 (C_5)_j^{(t-1)} \times_3 (C_6)_k^{(t-1)}\|_F)). \quad (17)$$

For W_E we have

$$(m_E)_{ijkl}^t = \max(\delta, \text{sqrt}(\|(\Lambda_E)^{(t-1)} \tilde{\mathcal{E}}_{ijkl}^n - \mathcal{G}_E \times_1 (P_2)_i^{(t-1)} \times_2 (P_3)_j^{(t-1)}\|_F)). \quad (18)$$

- (2) $\mathbf{C}_1, \mathbf{C}_2, \mathbf{C}_3, \mathbf{C}_4$: For the i th mode, let $C_{\sim i} = C_4 \otimes \dots \otimes C_{i+1} \otimes C_{i-1} \otimes \dots \otimes C_1$, and let $K = (\mathcal{G}_Q)_{(i)} C_{\sim i}^T$. Then, let x_j denote the j th row of the variable being optimized for. Then

$$x_j = (\frac{\rho}{2}(B - \Gamma_i)_j + (\frac{1}{n_Q}(W_{(i)}^2)_j \odot_b [(\Lambda \odot_b \tilde{\mathcal{Q}}^n)_{(i)}]_j) K^T) (\frac{\rho}{2} I_{4 \times 4} + \frac{1}{n_Q} K \text{diag}((W_{(i)}^2)_j) K^T)^{-1}. \quad (19)$$

- (3) $\mathbf{C}_5, \mathbf{C}_6, \mathbf{P}_1$: To solve for $C_i, i=5,6$, let $C_{\sim i} = P \otimes C_j$, and let $K = (\mathcal{G}_T)_{(i)} C_{\sim i}^T$. Then, we have the update rule for C_i as

$$x_j = (\frac{\rho}{2}(B - \Gamma_i)_j + \frac{1}{n_T} ((W_T^2 \odot L_T) \odot_b T)_j K^T) (\frac{\rho}{2} I_{4 \times 4} + \frac{1}{n_T} K \text{diag}((W_T^2)_j) K^T)^{-1}. \quad (20)$$

For P_1 , let $K = (\mathcal{G}_T)_{(3)} (C_2 \otimes C_1)^T$. Then

$$x_j = (\frac{\rho}{2}(D - \tau_i)_j + \frac{1}{n_T} ((W_T^2 \odot L_T) \odot_b T)_j K^T) (\frac{\rho}{2} I_{6 \times 6} + \frac{1}{n_T} K \text{diag}(W_T^2) K^T)^{-1}. \quad (21)$$

- (4) $\mathbf{P}_2, \mathbf{P}_3$: To solve for $P_i, i=2,3$, let $K = (\mathcal{G}_E)_{(i)} P_j^T$ where $j \neq i$. Then, we have the update rule for P_i as

$$x_j = (\frac{\rho}{2}(D - \tau_i)_j + \frac{1}{n_E} ((W_E^2 \odot L_E) \odot_b E)_j K^T) (\frac{\rho}{2} I_{6 \times 6} + \frac{1}{n_E} K \text{diag}(W_E^2) K^T)^{-1}. \quad (22)$$

- (5) $\Lambda_Q, \Lambda_T, \Lambda_E$: For Λ , we can solve as usual by directly solving the convex optimization problem associated with f_Q, g_T, h_E , This is done by solving for each block separately, where

$$(\Lambda_Q)_{ijkl} = \frac{\text{trace}((\llbracket \mathcal{G}_Q; C_1, C_2, C_3, C_4 \rrbracket_{ijkl})_{(1)}^T ((\tilde{\mathcal{Q}}^n)_{ijkl})_{(1)})}{\|((\tilde{\mathcal{Q}}^n)_{ijkl})_{(1)}\|_F^2}, \quad (23)$$

$$(\Lambda_T)_{ijk} = \frac{\text{trace}((\llbracket \mathcal{G}_T; P_1, C_5, C_6 \rrbracket_{ijk})_{(1)}^T ((\tilde{\mathcal{T}}^n)_{ijk})_{(1)})}{\|((\tilde{\mathcal{T}}^n)_{ijk})_{(1)}\|_F^2}, \quad (24)$$

$$(\Lambda_E)_{ij} = \frac{\text{trace}((\llbracket \mathcal{G}_E; P_2, P_3 \rrbracket_{ij})_{(1)}^T ((\tilde{\mathcal{E}}^n)_{ij})_{(1)})}{\|((\tilde{\mathcal{E}}^n)_{ij})_{(1)}\|_F^2}. \quad (25)$$

After calculating $\Lambda_Q, \Lambda_T, \Lambda_E$, for each of them, we first symmetrize, then normalize Λ so that $\|\Lambda\|_F^2 = 1$ and Λ satisfies the required symmetry requirements.

- (6) B, D : For B , we can solve directly, where

$$B = \frac{1}{6} \left(\sum_{i=1}^6 C_i + \Gamma_i \right). \quad (26)$$

For D , we also solve directly, where

$$D = \frac{1}{3} \left(\sum_{i=1}^3 P_i + \tau_i \right). \quad (27)$$

- (7) Γ_i, τ_i : For the final ascent cases for Γ_i, τ_i , we set

$$\Gamma_i = \Gamma_i + (C_i - B), \quad (28)$$

$$\tau_i = \tau_i^{(k)} + (P_i - D). \quad (29)$$

We note that in the joint optimization, we only constrain the equality between C_i 's and P_i 's. However, there is also a relationship between C_i and P_i , where P_i 's elements should be the 2×2 minors of C_i . In future work, this constraint could be further explored to potentially improve the algorithm and the linkage between the factor matrices and the three block entities.

10. Additional Experiments

10.1. Randomized Updates in QuadSync

We test the effect of randomized updates for C_i in QuadSync for the ETH3D ‘relief’ dataset. Our ‘relief’ dataset contains 13 images, and the maximum number of columns for updating C_i is $27 \times 13^3 = 59319$. We try randomized updates using columns whose number range from 20 to 59319. Using 30 random columns achieve similar accuracy with notable speed-ups. See Figure 4 for the effect of the number of columns on accuracy and runtime.

10.2. Collinear Experiments

Given a set of images, it is well known that the locations can only be uniquely determined up to a global transformation, only when the graph is *parallel rigid*, see [54] and [55] for more details on parallel rigidity in computer vision. The most degenerate case is when all of the cameras are collinear, meaning that they lie on a common line. In this case, synchronization algorithms that involve solving the location synchronization problem using solely pairwise directions, will fail. However, collinear or close to collinear situations are common in real life, such as in self-driving cars or robot motion.

Algorithm 2 Joint Opt. IRLS-ADMM

Input: $\mathcal{Q}^n \in \mathbb{R}^{3n \times 3n \times 3n \times 3n}$, $\mathcal{T}^n \in \mathbb{R}^{3n \times 3n \times 3n}$, $\rho > 0$ in $\mathbb{R}$
 $\mathcal{E}^n \in \mathbb{R}^{3n \times 3n \times 3n}$, Ω observed indices

Output: $\bar{C} \in \mathbb{R}^{3n \times 4}$

Normalize $\mathcal{Q}^n$, $\mathcal{T}^n$, $\mathcal{E}^n$ so that each block has norm 1

Calculate n_Q , n_T , n_E as the number of estimated blocks respectively.

Obtain C_i , B from first four singular vectors in first factor matrix of HOSVD($\mathcal{Q}^n$)

Calculate initial IRLS weights W_Q , W_T , W_E from (16), (17), (18).

while not converged **do**

Now we run the ADMM steps

while not converged **do**

First optimize for C_i , P_i , Λ_Q , Λ_T , Λ_E alternately

while not converged **do**

 Update all C_i , P_j , for $j=1,2,3$ via (19), (20), (21), (22)

 Update Λ_Q , Λ_T , Λ_E via (23), (24), (25)

end while

 Update B via (26)

 Update D via (27)

 Update Γ_i via (28)

 Update τ_i via (29)

end while

 Update new IRLS weights W_Q , W_T , W_E via (16), (17), (18)

end while

$\bar{C} = \text{avg}(C_1, C_2, C_3, C_4)$

Return

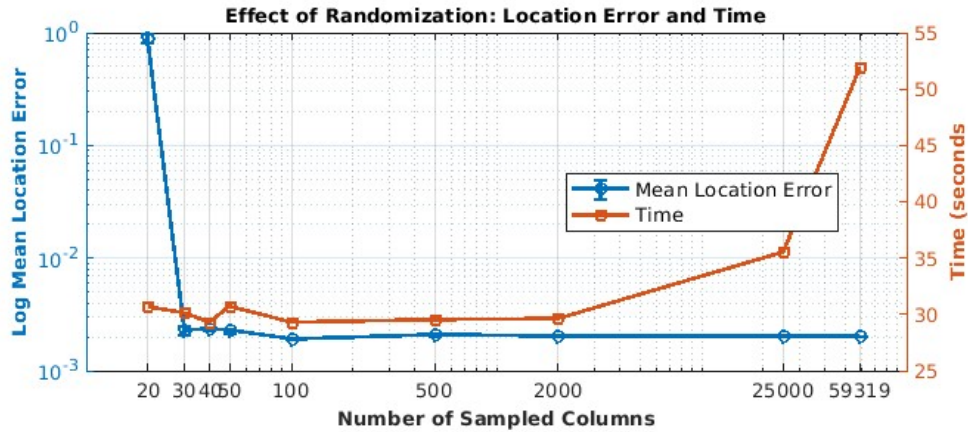


Figure 4. Randomized updates in QuadSync tested on ETH3D ‘relief’ dataset.

Ground Truth Location of 10 Collinear Cameras

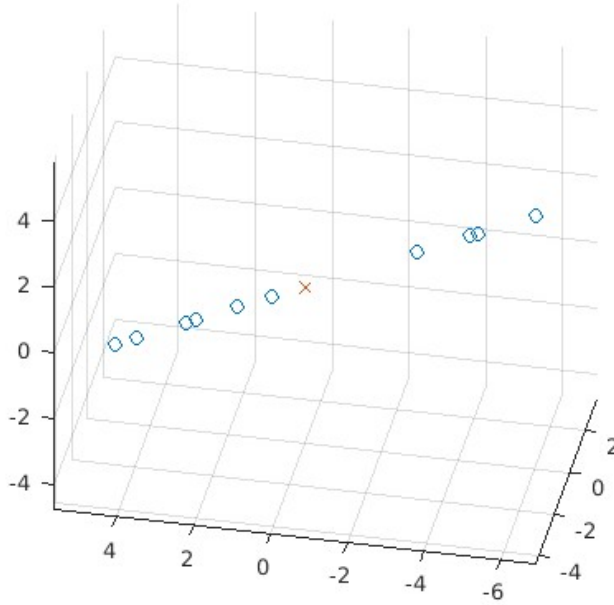


Figure 5. Ground truth location of 10 cameras. $\times$ is the origin. $\circ$ denotes the camera centers.

We conduct a small set of synthetic experiments to demonstrate that our algorithm can successfully recover camera poses in the collinear setting. A simple diagram can show the situation, see Figure 5.

We randomly generate calibrated collinear cameras. Then, for each individual quadrifocal tensor estimation, we add random noise of a certain percentage to its corresponding cameras and form the corresponding block quadrifocal tensor individually for each quadruplet. The noise parameter corresponds to the amount of noise we add to the cameras. We then randomly sample a certain percentage of quadruplets. We discard all the trifocal tensor and fundamental matrices. The results are then aggregated in Table 1. Our algorithm can still recover the camera parameters effectively in the collinear setting.

perc. (%)	noise(%)	$\bar{e}_t$	$\hat{e}_t$	$\bar{e}_r$	$\hat{e}_r$
100	0.00	0.00	0.00	0.00	0.00
100	1	0.04	0.04	0.23	0.18
100	5	0.24	0.22	2.67	2.52
80	0	0.00	0.00	0.00	0.00
80	1	0.04	0.03	0.37	0.37
80	5	0.36	0.36	3.07	2.68
60	0	0.00	0.00	0.00	0.00
60	1	0.06	0.05	0.42	0.38
60	5	0.64	0.54	4.52	3.67

Table 1. Results for synthetic experiments with collinear cameras. $\bar{e}_t$: mean location error. $\hat{e}_t$: median location error. $\bar{e}_r$: mean rotation error. $\hat{e}_r$: median rotation error.

Standard pipelines that synchronization rotation and location separately will fail in this situation. Global synchronization algorithms using the block essential matrix and the block trifocal tensors will need special tuning, as the ranks of the entities drop. The block quadrifocal tensor is insensitive to such cases and has a special advantage in the collinear motion setting.

10.3. Distributed Synchronization

Our quadrifocal tensor synchronization algorithm becomes slow as it deals with higher order tensors. However, it can be sped up with distributed synchronization, where clusters are formed and solved in parallel. In our set of experiments, we follow the way we add noise to the cameras in the collinear experiments. We artificially handpick the clusters, so that each cluster is fully dense (we continue to disregard trifocal and fundamental matrix elements). There are overlapping cameras in each cluster, and we finally synchronize all the clusters by calculating the projective transformation that maps the overlapping cameras to be in the same projective frame. The main take away for this experiment is to demonstrate the potential for applicability of the algorithm on large datasets. We conduct sets of experiments with just 2 CPU cores, and show that by focusing on smaller subsets of the data and using parallelization, we can greatly reduce the runtime of our algorithm, making it more scalable and applicable to larger scenes.

Experiment: We compare just the difference in runtime for a small synthetic dataset. We add no noise and assume that the entire block is observed accurately. We break the entire set of 30 cameras into 3 clusters. The second and first cluster have 5 overlapping cameras, same with the second and third. We then synchronize each cluster separately. The resulting cameras will be in different projective frames. We then use the overlapping cameras to align the clusters to the same projective frame. Noise is added similarly as in the synthetic collinear experiments. We include the results in Table 2.

We also include a small experiment with handpicked clusters for the EPFL CastleP30 dataset. Note that we can not directly synchronization CastleP30 using QuadSync, as the cameras point outwards around a courtyard, so that each camera only sees a small portion of the courtyard, and the higher-order viewing graph is very sparse. However, applying QuadSync on each of the handpick clusters then merging the results produces great reconstruction for the CastleP30 dataset. See Figure 6 for a comparison of the retrieved poses and ground truth poses.

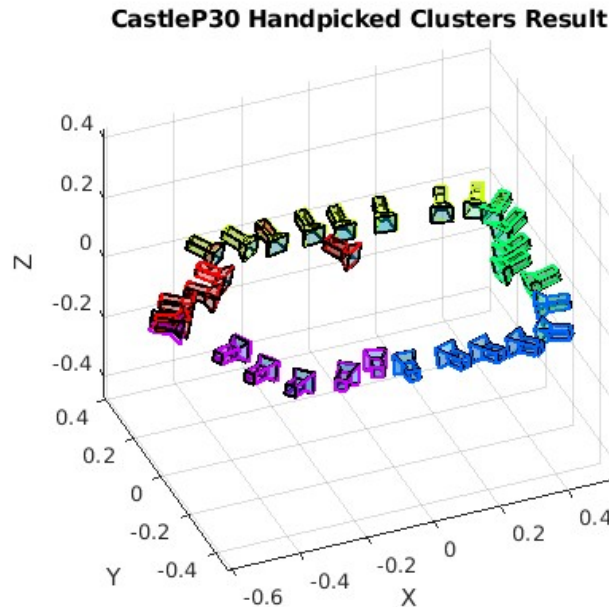


Figure 6. Retrieved poses (colored, where each cluster has a distinct color) vs. ground truth poses (black) for CastleP30 with handpicked clusters.

We note that we added uniform noise to the quadrifocal tensor estimates. Yet, the algorithm performs better in the corrupted setting, where some estimates are very accurate, and a smaller portion is very inaccurate. The main point that we would to demonstrate is that the algorithm can be significantly sped up when the viewing graph has obvious clusters without sacrificing too much accuracy. It is clear that the algorithm can be even further sped up when given more cores and computing resources.

size	noise	$\bar{e}_t$	$\hat{e}_t$	$\bar{e}_r$	$\hat{e}_r$	time(s)
Full (30 cams)	0	0	0	0	0	1666.3
Cluster1 (10 cams)	0	0	0	0	0	27.3
Cluster2 (15 cams)	0	0	0	0	0	92.6
Cluster3 (15 cams)	0	0	0	0	0	97.7
Aligned (30 cams)	0	0	0	0	0	150.2
Full (30 cams)	1	0.11	0.10	0.08	0.09	1944.8
Cluster1 (10 cams)	-	0.66	0.63	0.21	0.29	31.9
Cluster2 (15 cams)	-	0.31	0.32	0.18	0.14	111.5
Cluster3 (15 cams)	-	0.31	0.31	0.22	0.21	93.4
Aligned (30 cams)	-	0.53	0.44	0.35	0.32	247.5
Full (30 cams)	2	0.28	0.28	0.18	0.19	1776.0
Cluster1 (10 cams)	-	1.57	1.63	1.00	0.79	28.9
Cluster2 (15 cams)	-	0.77	0.76	0.41	0.36	112.2
Cluster3 (15 cams)	-	0.63	0.59	0.46	0.40	92.9
Aligned (30 cams)	-	1.14	0.97	0.74	0.62	245.7

Table 2. Results for synthetic distributed synchronization. $\bar{e}_t$: mean location error. $\hat{e}_t$: median location error. $\bar{e}_r$: mean rotation error. $\hat{e}_r$: median rotation error. Noise is added as percentage onto the cameras. In the noisy cases, only the quadrifocal tensors were synchronized.

11. Detailed Numerical Results

In this section, we include more comprehensive results accompanying the results in the main paper. We report the completion rates and an idea of the runtime in Table 3. We also report the detailed mean location error, median location error, mean rotation error, median rotation error in Tables 4, 5, 6, 7 respectively. Locations are reported in the metrics of their original datasets, and rotations are reported in degrees.

Table 3. Completion rates and runtimes of different methods

Dataset	Completion %	Joint Opt. (s)	TrifocalSync (s)
courtyard (18/38)	33.10	111.2	137.9
electro (12/39)	78.70	30.6	45.13
facade (46/76)	61.98	4226.2	1633.0
kicker (20/30)	31.84	157.3	174.3
meadow (6/14)	42.13	11.7	9.1
office (13/21)	39.92	34.2	53.3
pipes (8/14)	58.79	21.6	27.2
relief_2 (16/20)	14.31	64.9	121.6
relief (13/13)	95.21	36.3	75.4
terrace (11/23)	92.53	27.5	54.8
terrains (23/42)	8.29	223.1	238.5
CastleP19 (13/19)	24.46	45.2	77.24
CastleP30 (19/30)	28.02	139.7	174.7
EntryP10 (10/10)	26.50	25.0	41.8
FountainP11 (11/11)	76.55	22.8	47.3
HerzP8 (8/8)	80.76	17.6	26.7
HerzP25 (24/25)	25.38	312.7	312.8

Table 4. Mean location error by method

Dataset	QuadSync	Joint Opt.	Trifocal Sync	LUD	NRFM	MPLS BATA	MPLS CS
courtyard (18/38)	0.0477	0.0489	0.1753	0.0403	0.1104	0.0711	0.0234
electro (12/39)	0.0199	0.0200	0.0197	0.0605	0.0257	0.0617	0.0604
facade (46/76)	3.0294	0.0248	0.0222	0.2497	0.2068	0.0984	0.0158
kicker (20/30)	0.0078	0.0078	0.0204	0.0366	0.0246	0.1934	0.0178
meadow (6/14)	0.0401	0.1071	1.4341	0.6408	0.2207	1.8229	0.0691
office (13/21)	0.0039	0.0037	0.0151	0.0262	0.0187	0.0166	0.0086
pipes (8/14)	0.0192	0.0115	0.0344	0.1097	0.1058	0.1379	0.0085
relief_2 (16/20)	0.5210	0.5360	0.5554	0.8624	0.8526	0.2547	0.0158
relief (13/13)	0.0020	0.0020	0.0020	0.0043	0.0047	0.1076	0.0052
terrace (11/23)	0.0096	0.0096	0.0103	0.0165	0.0150	0.0163	0.0147
terrains (23/42)	0.4631	0.4335	0.0797	0.0092	0.0149	0.0326	0.0075
CastleP19 (13/19)	0.5130	0.4921	7.6932	0.2605	0.5965	1.3871	0.1094
CastleP30 (19/30)	0.1629	0.1627	6.4738	0.1132	0.1683	0.4945	0.0687
EntryP10 (10/10)	0.0007	0.0007	0.0654	0.0957	0.2913	0.1235	0.0897
FountainP11 (11/11)	0.0002	0.0002	0.0098	0.0103	0.0223	0.2125	0.0095
HerzP25 (24/25)	0.0025	0.0026	0.2187	0.0647	0.1114	9.2329	0.1668
HerzP8 (8/8)	0.0010	0.0010	0.0180	0.0522	0.2299	0.5766	0.0375

Table 5. Median location error by method

Dataset	QuadSync	Joint Opt.	Trifocal Sync	LUD	NRFM	MPLS BATA	MPLS CS
courtyard	0.0307	0.0324	0.0947	0.0308	0.0668	0.0598	0.0194
electro	0.0151	0.0152	0.0133	0.0256	0.0081	0.0221	0.0181
facade	2.5853	0.0135	0.0140	0.0108	0.0124	0.0253	0.0128
kicker	0.0068	0.0063	0.0173	0.0090	0.0110	0.0788	0.0101
meadow	0.0390	0.0922	1.1006	0.2374	0.1285	0.7635	0.0358
office	0.0028	0.0027	0.0090	0.0035	0.0047	0.0043	0.0033
pipes	0.0139	0.0075	0.0212	0.0390	0.0506	0.1330	0.0046
relief_2	0.4392	0.4737	0.5204	0.7633	0.7852	0.2065	0.0072
relief	0.0017	0.0017	0.0017	0.0048	0.0048	0.0052	0.0049
terrace	0.0091	0.0091	0.0104	0.0173	0.0172	0.0164	0.0159
terrains	0.4442	0.4052	0.0609	0.0065	0.0114	0.0165	0.0070
CastleP19	0.5502	0.4944	7.2040	0.1694	0.4583	1.1433	0.0913
CastleP30	0.1145	0.1169	5.6921	0.0698	0.1536	0.1670	0.0485
EntryP10	0.0006	0.0006	0.0514	0.0851	0.3107	0.1058	0.0571
FountainP11	0.0002	0.0002	0.0079	0.0086	0.0166	0.1889	0.0085
HerzP25	0.0009	0.0010	0.1876	0.0463	0.0849	0.1525	0.0608
HerzP8	0.0007	0.0007	0.0147	0.0545	0.0808	0.4254	0.0358

Table 6. Mean rotation error by method

Dataset	QuadSync	Joint Opt.	Trifocal Sync	LUD	NRFM	MPLS
courtyard	0.3059	0.3295	1.2398	0.1886	0.1886	0.1056
electro	0.1022	0.1020	0.1208	0.0722	0.0722	0.0603
facade	2.1185	0.1131	0.1239	0.0442	0.0442	0.0506
kicker	0.2063	0.2028	0.2986	0.1162	0.1162	0.2697
meadow	0.2856	0.6100	12.4079	0.1403	0.1403	0.1861
office	0.1653	0.1449	0.5749	0.0674	0.0674	0.0547
pipes	0.5901	0.1785	3.0799	0.1516	0.1516	0.1546
relief_2	32.6666	49.3035	52.5503	45.0175	45.0175	0.1049
relief	0.0791	0.0791	0.0792	0.1080	0.1080	0.1077
terrace	0.0796	0.0796	0.0812	0.0728	0.0728	0.0693
terrains	11.5185	11.1041	3.7660	0.2027	0.2027	0.1918
CastleP19	30.4922	31.9495	39.5514	0.7604	0.7604	0.2795
CastleP30	15.8584	14.8579	10.6018	0.1937	0.1937	0.1168
EntryP10	0.0822	0.0822	0.1489	0.2993	0.2993	0.3086
FountainP11	0.0734	0.0734	0.0912	0.0517	0.0517	0.0508
HerzP25	0.2271	0.2300	1.1558	0.2879	0.2879	0.7712
HerzP8	0.0997	0.0997	0.1167	0.4717	0.4717	0.3462

Table 7. Median rotation error by method

Dataset	QuadSync	Joint Opt.	Trifocal Sync	LUD	NRFM	MPLS
courtyard	0.2144	0.2283	1.2651	0.1909	0.1909	0.1234
electro	0.0883	0.0888	0.1186	0.0630	0.0630	0.0638
facade	0.9737	0.0731	0.0812	0.0449	0.0449	0.0457
kicker	0.1735	0.1701	0.2877	0.1146	0.1146	0.1650
meadow	0.2002	0.5049	7.3067	0.0902	0.0902	0.1337
office	0.0994	0.1070	0.2807	0.0717	0.0717	0.0703
pipes	0.2937	0.1131	1.7915	0.1310	0.1310	0.1312
relief_2	7.9110	12.0705	19.4790	38.7239	38.7239	0.1039
relief	0.0880	0.0880	0.0849	0.1173	0.1173	0.1157
terrace	0.0860	0.0860	0.0885	0.0962	0.0962	0.0916
terrains	2.6535	2.3353	2.6054	0.2280	0.2280	0.2567
CastleP19	6.7486	8.3131	7.7256	0.5380	0.5380	0.2186
CastleP30	7.9894	7.8164	4.2029	0.1520	0.1520	0.0903
EntryP10	0.0810	0.0810	0.1173	0.2840	0.2840	0.2375
FountainP11	0.0724	0.0724	0.0884	0.0559	0.0559	0.0448
HerzP25	0.1500	0.1584	0.7872	0.2564	0.2564	0.4064
HerzP8	0.0813	0.0813	0.1107	0.4824	0.4824	0.3354